

Professional and High-Performance C++

Build Systems, Testing, Debugging, Architecture,
Profiling, and CPU Optimization

Dr. Charles Dorner

2026

Educational manuscript and companion sources provided to the recipient. Third-party tools and libraries retain their own licenses.

Professional and High-Performance C++

1. [About this volume](#)
2. [How to use this volume](#)
 1. [Learn from contracts, not isolated syntax](#)
 2. [Build one chapter or the complete companion](#)
 3. [Verification and portability boundaries](#)
 4. [Use evidence to close the learning loop](#)
3. [Chapter 1 — Structuring Production C++ Repositories](#)
 1. [1.1 Organizing Source Code by Responsibility](#)
 1. [1.1.1 Separating Libraries from Applications](#)
 2. [1.1.2 Separating Public and Private Interfaces](#)
 2. [1.2 Managing Configuration and Build Artifacts](#)
 1. [1.2.1 Separating Source and Build Trees](#)
 2. [1.2.2 Managing Generated Files](#)
 3. [Worked example — Separate a measurement library from its executable](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
4. [Chapter 2 — Building Projects with CMake](#)
 1. [2.1 Defining Target-Based Builds](#)
 1. [2.1.1 Creating Executable Targets](#)
 2. [2.1.2 Creating Library Targets](#)
 2. [2.2 Managing Target Dependencies](#)
 1. [2.2.1 Propagating Include Directories](#)
 2. [2.2.2 Propagating Compile Definitions and Options](#)
 3. [Worked example — Configure and test a standalone target graph](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)

5. [Chapter 3 — Understanding Static and Dynamic Linking](#)
 1. [3.1 Building Static Libraries](#)
 1. [3.1.1 Archiving Object Files](#)
 2. [3.1.2 Linking Static Dependencies](#)
 2. [3.2 Building Shared Libraries](#)
 1. [3.2.1 Understanding Dynamic Loading](#)
 2. [3.2.2 Managing Symbol Visibility](#)
 3. [Worked example — Build the same contract as static and shared libraries](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
6. [Chapter 4 — Managing Third-Party Dependencies](#)
 1. [4.1 Choosing Dependency Strategies](#)
 1. [4.1.1 Vendoring Source Dependencies](#)
 2. [4.1.2 Using Package Managers](#)
 2. [4.2 Making Builds Reproducible](#)
 1. [4.2.1 Pinning Dependency Versions](#)
 2. [4.2.2 Isolating Build Environments](#)
 3. [Worked example — Verify an offline vendored helper](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
7. [Chapter 5 — Managing C++ Development with Git](#)
 1. [5.1 Organizing Changes into Commits](#)
 1. [5.1.1 Creating Focused Commits](#)
 2. [5.1.2 Writing Useful Commit Messages](#)
 2. [5.2 Collaborating Through Branches and Reviews](#)
 1. [5.2.1 Managing Feature Branches](#)
 2. [5.2.2 Resolving Merge Conflicts](#)
 3. [Worked example — Review a parser change and rehearse a branch safely](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)

4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
8. [Chapter 6 — Designing Effective Unit Tests](#)
 1. [6.1 Isolating Small Units of Behavior](#)
 1. [6.1.1 Structuring Test Cases](#)
 2. [6.1.2 Defining Expected Outcomes](#)
 2. [6.2 Testing Edge and Failure Cases](#)
 1. [6.2.1 Testing Boundary Conditions](#)
 2. [6.2.2 Testing Error Paths](#)
 3. [Worked example — Build a table-driven parser test](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
9. [Chapter 7 — Designing Integration and System Tests](#)
 1. [7.1 Testing Component Interactions](#)
 1. [7.1.1 Verifying Interface Contracts](#)
 2. [7.1.2 Testing Real Dependencies](#)
 2. [7.2 Testing Complete Applications](#)
 1. [7.2.1 Designing End-to-End Scenarios](#)
 2. [7.2.2 Capturing Reproducible Failures](#)
 3. [Worked example — Round-trip a real measurement file](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
10. [Chapter 8 — Isolating Dependencies with Test Doubles](#)
 1. [8.1 Understanding Stubs, Fakes, and Mocks](#)
 1. [8.1.1 Replacing Expensive Dependencies](#)
 2. [8.1.2 Controlling Test Inputs](#)
 2. [8.2 Designing Code for Testability](#)
 1. [8.2.1 Injecting Dependencies](#)
 2. [8.2.2 Avoiding Hidden Global Dependencies](#)
 3. [Worked example — Test cache expiration without sleeping](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)

3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
11. [Chapter 9 — Debugging Native C++ Applications](#)
 1. [9.1 Controlling Programs with GDB or LLDB](#)
 1. [9.1.1 Setting Breakpoints and Watchpoints](#)
 2. [9.1.2 Inspecting Variables and Stack Frames](#)
 2. [9.2 Diagnosing Postmortem Failures](#)
 1. [9.2.1 Reading Stack Traces](#)
 2. [9.2.2 Analyzing Core Dumps](#)
 3. [Worked example — Inspect a deterministic calculation with a debugger](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
12. [Chapter 10 — Detecting Memory and Undefined-Behavior Defects](#)
 1. [10.1 Using Compiler Sanitizers](#)
 1. [10.1.1 Detecting Invalid Memory Access](#)
 2. [10.1.2 Detecting Undefined Behavior](#)
 2. [10.2 Detecting Resource Leaks](#)
 1. [10.2.1 Identifying Leaked Allocations](#)
 2. [10.2.2 Tracking Resource Ownership](#)
 3. [Worked example — Check cleanup and isolate a negative sanitizer fixture](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
13. [Chapter 11 — Improving Code with Static Analysis](#)
 1. [11.1 Treating Compiler Warnings as Engineering Signals](#)
 1. [11.1.1 Enabling Strong Warning Levels](#)
 2. [11.1.2 Removing Warning Debt](#)
 2. [11.2 Applying Dedicated Static-Analysis Tools](#)
 1. [11.2.1 Detecting Correctness Problems](#)
 2. [11.2.2 Enforcing Coding Guidelines](#)
 3. [Worked example — Make an indexing contract analyzable](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)

3. [Run and verify](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
14. [Chapter 12 — Measuring Performance with Benchmarks](#)
 1. [12.1 Designing Reliable Microbenchmarks](#)
 1. [12.1.1 Preventing Compiler Elimination](#)
 2. [12.1.2 Measuring Repeated Operations](#)
 2. [12.2 Interpreting Benchmark Results](#)
 1. [12.2.1 Understanding Variance](#)
 2. [12.2.2 Comparing Implementations Fairly](#)
 3. [Worked example — Compare ordinary and four-lane reductions](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
15. [Chapter 13 — Profiling CPU Performance](#)
 1. [13.1 Locating Runtime Hotspots](#)
 1. [13.1.1 Measuring Function-Level CPU Time](#)
 2. [13.1.2 Inspecting Call Graphs](#)
 2. [13.2 Understanding Hardware Performance Counters](#)
 1. [13.2.1 Measuring Cache Misses](#)
 2. [13.2.2 Measuring Branch Behavior](#)
 3. [Worked example — Profile equivalent lookup strategies](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
16. [Chapter 14 — Understanding Compiler Optimization](#)
 1. [14.1 Understanding Optimization Levels](#)
 1. [14.1.1 Comparing Debug and Optimized Builds](#)
 2. [14.1.2 Understanding Inlining and Constant Folding](#)
 2. [14.2 Inspecting Generated Machine Code](#)
 1. [14.2.1 Reading Compiler Assembly Output](#)
 2. [14.2.2 Verifying Optimization Assumptions](#)

3. [Worked example — Compare two defined formulations before inspecting assembly](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
17. [Chapter 15 — Understanding CPU Architecture for C++ Performance](#)
 1. [15.1 Understanding Modern Processor Pipelines](#)
 1. [15.1.1 Understanding Instruction-Level Parallelism](#)
 2. [15.1.2 Understanding Out-of-Order Execution](#)
 2. [15.2 Understanding Memory Hierarchies](#)
 1. [15.2.1 Understanding CPU Cache Levels](#)
 2. [15.2.2 Understanding Main-Memory Latency](#)
 3. [Worked example — Expose independent reduction chains](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
18. [Chapter 16 — Designing Cache-Friendly Software](#)
 1. [16.1 Improving Data Locality](#)
 1. [16.1.1 Improving Spatial Locality](#)
 2. [16.1.2 Improving Temporal Locality](#)
 2. [16.2 Choosing Better Data Layouts](#)
 1. [16.2.1 Comparing Arrays of Structures](#)
 2. [16.2.2 Comparing Structures of Arrays](#)
 3. [Worked example — Convert records to validated columns](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
19. [Chapter 17 — Accelerating Computation with SIMD](#)
 1. [17.1 Understanding Vector Execution](#)
 1. [17.1.1 Processing Multiple Values per Instruction](#)
 2. [17.1.2 Understanding SIMD Register Widths](#)

2. [17.2 Enabling Vectorization](#)
 1. [17.2.1 Writing Auto-Vectorizable Loops](#)
 2. [17.2.2 Using Explicit SIMD Intrinsics When Necessary](#)
3. [Worked example — Add vectors with a guarded SIMD implementation](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
20. [Chapter 18 — Applying Data-Oriented Design](#)
 1. [18.1 Designing Around Data Transformations](#)
 1. [18.1.1 Grouping Frequently Processed Data](#)
 2. [18.1.2 Eliminating Pointer-Chasing Structures](#)
 2. [18.2 Comparing Object-Oriented and Data-Oriented Models](#)
 1. [18.2.1 Evaluating Abstraction Costs](#)
 2. [18.2.2 Selecting Layouts Based on Workloads](#)
 3. [Worked example — Materialize an active processing set](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
21. [Chapter 19 — Engineering High-Performance Concurrency](#)
 1. [19.1 Reducing Synchronization Costs](#)
 1. [19.1.1 Reducing Lock Contention](#)
 2. [19.1.2 Avoiding False Sharing](#)
 2. [19.2 Designing Scalable Work Distribution](#)
 1. [19.2.1 Building Thread Pools](#)
 2. [19.2.2 Applying Work Partitioning](#)
 3. [Worked example — Build a bounded future-returning task pool](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
22. [Chapter 20 — Designing Stable Libraries and APIs](#)
 1. [20.1 Designing Clear Public Interfaces](#)
 1. [20.1.1 Minimizing Coupling](#)

2. [20.1.2 Preserving Encapsulation](#)
2. [20.2 Managing Binary Compatibility](#)
 1. [20.2.1 Understanding ABI Concerns](#)
 2. [20.2.2 Applying the Pimpl Idiom](#)
3. [Worked example — Hide accumulator storage behind Pimpl](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
23. [Chapter 21 — Building, Testing, Profiling, and Shipping a Production System](#)
 1. [21.1 Establishing the Production Build Pipeline](#)
 1. [21.1.1 Automating Compilation and Tests](#)
 2. [21.1.2 Producing Release Artifacts](#)
 2. [21.2 Optimizing Based on Evidence](#)
 1. [21.2.1 Establishing Performance Baselines](#)
 2. [21.2.2 Verifying Measured Improvements](#)
 3. [Worked example — Build and package HarborMetrics](#)
 1. [Trace the implementation](#)
 2. [Complete worked source](#)
 3. [Run and verify](#)
 4. [Practice and implementation lab](#)
 1. [Solution and review notes](#)
24. [Glossary](#)
25. [Shared source reference](#)
 1. [check.hpp](#)
 2. [bounded_queue.hpp](#)
26. [Technical reference directory](#)

About this volume

A program becomes a product when someone other than its original author can build it, verify its behavior, diagnose a failure, and operate the delivered artifact. This volume follows HarborMetrics, a small measurement-processing application, from repository organization through release evidence. The examples are deliberately small enough to inspect, but their engineering questions are the same ones that appear in much larger systems.

The default examples use C++20 and the shared CMake build. Optional standalone projects demonstrate library linking, dependency checks, and packaging without changing the main repository. Timing programs print observations, not universal speedup promises. Run performance experiments in an optimized build after correctness tests, record the machine and workload, and do not compare sanitizer timings with ordinary release timings.

Read the final verification report for the exact compiler and execution coverage obtained for this edition. A local test result is evidence for those inputs and that configuration, not a certification of every platform. The workflows and scripts are supplied so the same questions can be asked again on a different machine.

How to use this volume

Learn from contracts, not isolated syntax

Read the numbered concepts first, then follow the worked example. Each chapter includes complete source files, an explanation of the mechanism, a practice task, and solution notes. The examples are teaching programs with explicit scope; they are not a promise of production certification. Run the supplied checks before modifying an example, then add a failing case that captures the behavior you intend to change.

The six books progress from language foundations to modern ownership, algorithms, systems interfaces, professional tooling, and heterogeneous computing. Chapter numbering restarts in each book. The companion keeps the same hierarchy: `source/book01/ch01/` through `source/book06/ch22/`. Every chapter has its own CMake target and fixture tests. Applications use the fictional Harbor naming family, not an employer's infrastructure or confidential code.

Build one chapter or the complete companion

The primary CPU source baseline is C++20. The CUDA translation units use a deliberate C++17 subset at their host/device boundary. CMake 3.24 or later, a compatible compiler and standard library, Python 3.10 or later, and a build tool are required. Installation guidance is in Book I; platform-specific procedures are explicitly labeled. No third-party download is required by the default CPU examples.

From the extracted ZIP's package directory:

```
cmake -S source -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 4
ctest --test-dir build --output-on-failure
```

To work on one chapter, replace the two numbers below with the book and chapter:

```
python3 tools/run_chapter.py 1 1
```

The test runner executes each fixture in a fresh temporary working directory. File-writing examples use that isolated directory. Do not point experiments at valuable files or run the companion as an administrator. Intentional defect examples under `optional/` are excluded from the normal build. Read their local warnings before running them separately.

Verification and portability boundaries

This edition includes the actual `TEST_REPORT.md` and build logs in the ZIP. CPU tests were run on the recorded Linux environment; those finite checks are not exhaustive proofs. The report distinguishes compiler runs, sanitizers, optional tooling, document checks, and unsupported environments.

No CUDA compiler or NVIDIA GPU was available in the publishing environment. CUDA source is provided with checked runtime calls, CPU references, and a target-machine validation procedure, but it has not been compiled or executed here. CPU success must never be reported as a GPU pass. Windows, macOS, multiple GPUs, hard-real-time scheduling, and hardware-specific performance are not validated by the Linux CPU tests. Observed timings in runnable examples are measurements of the reader's run; hypothetical models are explicitly labeled.

The PDFs are reading editions at 7×10 inches, not automatically approved print-on-demand interiors. The EPUBs are reflowable; long code lines may wrap on small screens. Copy executable source from the companion, not from a visually wrapped PDF. No executable is embedded in the EPUB.

Use evidence to close the learning loop

Keep the input, expected result, observed result, compiler version, flags, and platform with each experiment. A correctness check and a benchmark answer different questions. A passing sanitizer run does not prove the absence of every defect; a faster timing does not excuse an invalid result. Chapter solutions explain the reasoning rather than merely giving an answer string.

Chapter 1 — Structuring Production C++ Repositories

Repository layout should make dependencies and ownership easier to discover. HarborMetrics begins with a library that summarizes bounded integer measurements and a tiny executable that exercises its contract. The important separation is not the number of folders: it is whether the computational library can be built and tested without inheriting command-line parsing, global logging, or a particular deployment environment.

1.1 Organizing Source Code by Responsibility

1.1.1 Separating Libraries from Applications

A library exposes operations that other code can call; an application chooses a policy for invoking them. Put a reusable summarize operation in the library and leave argument interpretation, terminal output, and exit codes in the executable. The library should report a result or a documented error rather than terminating the process itself.

This boundary permits multiple clients: a command-line tool, a service adapter, and a unit test can use the same implementation. It also gives a reviewer a concrete question to ask: does this change belong to the computation's contract, or only to one user's workflow? Splitting every function into a library is unnecessary; split responsibilities that genuinely vary independently.

1.1.2 Separating Public and Private Interfaces

A public header is a promise to consumers. Include only the types and declarations they need, make it self-contained, and avoid exposing implementation-only containers or platform headers without a reason. Private helpers can remain in an implementation file or a private include directory.

A public dependency becomes a consumer dependency. If a header mentions a third-party type in its signature, users may need that dependency even when the library internally hides most of its work. Keep that consequence visible in both architecture and build metadata rather than relying on include paths accidentally inherited from a larger application.

Separating a Public Library Interface from Private Implementation

Purpose

The source tree is organized by responsibility: consumers see a stable public header; implementation stays in the library source directory, and the application depends only on the public contract.

Code or command

```
// include/calc/add.hpp
#pragma once
int add(int a, int b);

// src/add.cpp
#include "calc/add.hpp"
int add(int a, int b) { return a + b; }

// app/main.cpp
#include "calc/add.hpp"
#include <iostream>
int main() { std::cout << add(20, 22) << "\n"; }
```

What the important parts do

`add.hpp` is the public interface. `add.cpp` contains the implementation that can change without forcing callers to include private details. `main.cpp` demonstrates the application/library dependency direction.

Expected behavior or output

When built and linked correctly, the application prints `42`.

Concrete experiment

Move an implementation-only helper into the public header, then list which translation units must rebuild when that helper changes. Move it back to the `.cpp` file and compare.

1.2 Managing Configuration and Build Artifacts

1.2.1 Separating Source and Build Trees

Source files express the intended program; build files record a particular toolchain's realization of it. Use separate build directories for Debug, Release, and sanitizer configurations. Reusing one cache while switching compilers or incompatible options can conceal which configuration produced a binary.

An out-of-source build also makes cleanup safer. Removing a known build directory should not remove authored code. The supplied launcher places

generated artifacts outside chapter source directories. Review a path before deleting it, and never use a recursive removal command against an unresolved variable as a routine build instruction.

1.2.2 Managing Generated Files

Generated headers, protocol bindings, and version manifests need a declared producer and declared inputs. A developer should be able to reconstruct them from committed sources or understand why an exact generated snapshot is intentionally committed. Untracked generation steps become hidden prerequisites on a clean machine.

Distinguish reproducible content from build metadata that intentionally changes, such as timestamps. A build identifier can aid diagnosis, but embedding the current time everywhere defeats byte-for-byte comparison. The package's authoring data and rendering scripts are separate from its exported manuscripts for the same reason: changes have a traceable source.

Keeping Source, Generated Configuration, and Build Products Separate

Purpose

Out-of-source builds keep generated files, caches, object files, and executables out of the maintained source tree.

Code or command

```
mkdir -p project/{src,include,cmake}  
cmake -S project -B project/build -DCMAKE_BUILD_TYPE=Debug  
cmake --build project/build  
find project/build -maxdepth 2 -type f | head
```

What the important parts do

`-S project` identifies immutable project input while `-B project/build` selects a separate build tree. CMake writes `CMakeCache.txt`, generated build files, and compiled artifacts under that directory instead of beside source files.

Expected behavior or output

The build directory contains generated metadata and artifacts while the original `src`, `include`, and `cmake` directories remain source-owned.

Concrete experiment

Delete `project/build` and reconfigure with a different compiler or build type. Confirm that no generated state had to be manually removed from the source tree.

Worked example — Separate a measurement library from its executable

Trace the implementation

The header declares a small result type and one operation. It does not depend on the test helper or the console. The implementation establishes an arithmetic bound before adding each value: at most one million accepted samples of magnitude at most one million fit comfortably in long long. The mean is a derived floating-point result, not an exact rational type.

The executable checks both a successful calculation and the empty-input policy. Its CHECK macro remains active in optimized builds, unlike an assertion that may disappear under NDEBUG. The generated chapter CMake file makes summary.cpp a static library and links main.cpp against it. That real link boundary exercises the public declaration rather than recompiling a pasted function inside the test.

The chapter-sized layout keeps files adjacent for reading. A larger repository can move the public header to include/harbor, implementation to src, and tests to tests without changing the dependency direction. Folder names are secondary to that direction.

Complete worked source

The files below are in source/book05/ch01/. The source in this chapter and the companion are exported from the same authored data.

summary.hpp

```
#ifndef HARBOR_SUMMARY_HPP
#define HARBOR_SUMMARY_HPP
#include <cstdlib>
#include <span>
namespace harbor {
struct Summary { std::size_t count; long long total; double mean; };
Summary summarize(std::span<const int> values);
}
#endif
```

summary.cpp

```
#include "summary.hpp"
#include <stdexcept>
namespace harbor {
Summary summarize(std::span<const int> values) {
    if (values.empty() || values.size() > 1000000)
        throw std::invalid_argument("sample count");
    long long total = 0;
    for (int value : values) {
        if (value < -1000000 || value > 1000000)
            throw std::invalid_argument("sample magnitude");
        total += value;
    }
    return {values.size(), total, static_cast<double>(total)/
static_cast<double>(values.size())};
}
```

```
}  
}
```

main.cpp

```
#include "check.hpp"  
#include "summary.hpp"  
#include <array>  
#include <stdexcept>  
int main() {  
    const std::array values{2, 4, 6};  
    const auto result = harbor::summarize(values);  
    CHECK(result.count == 3 && result.total == 12 && result.mean == 4.0);  
    bool rejected = false;  
    try { (void)harbor::summarize({}); }  
    catch (const std::invalid_argument&) { rejected = true; }  
    CHECK(rejected);  
    std::cout << "PASS\n";  
}
```

.gitignore

```
/build*/  
*.o  
*.obj  
*.exe  
compile_commands.json
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 1
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a second executable that prints a summary, while preserving the library's independence from iostream. Compile a translation unit that includes only summary.hpp to verify that the header supplies its own required declarations.

Document which files are authored, generated, or temporary. Decide whether a generated version header belongs in the source archive or should be recreated during configuration.

Solution and review notes

The printing client owns output formatting and exit behavior. The header-only compilation probe should not depend on another header happening to include span or cstdint first. Such a probe catches transitive-include accidents that a large application can hide.

A generated version header should have one declared producer and stable inputs when reproducibility is required. If generated files are distributed for consumers without the generator, retain provenance and a regeneration check rather than maintaining two independent hand-edited versions.

Technical references: [CMAKE](#) [GUIDELINES](#) [ODR](#)

Chapter 2 — Building Projects with CMake

CMake describes targets and their requirements, then generates a build for a selected toolchain. This chapter avoids global include paths and compiler flags by expressing a small calibration library as a target. A standalone CMake project is included so the target relationships can be inspected without the series' larger build surrounding them.

2.1 Defining Target-Based Builds

2.1.1 Creating Executable Targets

An executable target names its source files and the libraries it consumes. CMake's configure step constructs the build graph; the build step invokes the selected compiler and linker. A successful configure does not prove the program compiles, and a successful build does not prove its runtime behavior.

Use `cmake -S` for the source directory and `-B` for the build directory, then `cmake --build`. Multi-configuration generators select Debug or Release at build time with `--config`; single-configuration generators commonly use `CMAKE_BUILD_TYPE` at configuration. Record that distinction in instructions rather than assuming one generator's output paths everywhere.

2.1.2 Creating Library Targets

A library target gives related implementation files a reusable build identity. `STATIC` creates an archive for later linking; `SHARED` creates a dynamically linked library; `INTERFACE` can carry header-only usage requirements without producing an ordinary compiled library artifact.

Target names are logical build names, not necessarily installed filenames. Prefer linking the logical target to typing a guessed archive path. That lets CMake order prerequisites, propagate requirements, and account for platform-specific naming. A target still needs a deliberate install/export policy before it becomes a consumable package.

Defining Independent Library and Executable Targets in CMake

Purpose

Target-based CMake models actual build products and lets each product carry the requirements consumers need.

Code or command

```
add_library(mathlib src/add.cpp)
target_include_directories(mathlib PUBLIC include)
target_compile_features(mathlib PUBLIC cxx_std_20)

add_executable(calculator app/main.cpp)
target_link_libraries(calculator PRIVATE mathlib)
```

What the important parts do

``add_library`` and ``add_executable`` create two distinct nodes in the build graph. The library publishes its include directory and C++ standard requirement; the executable links to the library instead of inheriting unrelated global flags.

Expected behavior or output

CMake generates build rules in which ``mathlib`` is built before ``calculator`` when needed.

Concrete experiment

Add a second executable that links ``mathlib``. Confirm it receives the public include path without duplicating ``include_directories`` globally.

2.2 Managing Target Dependencies

2.2.1 Propagating Include Directories

An include directory is `PUBLIC` when the target and its consumers need it, `PRIVATE` when only the implementation needs it, and `INTERFACE` when only consumers need the requirement. Public headers that include other public headers must have their search requirements available to consumers.

For installed packages, build-tree paths and install-tree paths differ. `BUILD_INTERFACE` and `INSTALL_INTERFACE` generator expressions can describe that distinction. Hard-coding an author's absolute source directory in an exported package makes it appear to work locally while failing after relocation.

2.2.2 Propagating Compile Definitions and Options

Compile features express a language requirement such as `cxx_std_20`. Definitions should propagate only when consumers must see the same interface-relevant setting. An implementation's warning preferences are normally private; a macro changing a public class layout is an ABI concern that cannot safely vary between producer and consumer.

Keep compiler-specific options behind compiler checks. A Unix warning flag is not automatically meaningful to MSVC. Treat sanitizer and link options as a coordinated configuration, and inspect verbose build output when a property seems not to reach the intended target. Build metadata is executable engineering policy, not decoration around the source.

Propagating PUBLIC and PRIVATE Target Requirements

Purpose

The point of target dependencies is not just link order; usage requirements must propagate only when consumers actually need them.

Code or command

```
add_library(core src/core.cpp)
target_include_directories(core PUBLIC include PRIVATE src)
target_compile_definitions(core PRIVATE CORE_BUILD=1)

add_executable(app app/main.cpp)
target_link_libraries(app PRIVATE core)
```

What the important parts do

``PUBLIC include`` is visible both while building ``core`` and while compiling consumers. ``PRIVATE src`` and ``CORE_BUILD=1`` affect only the library implementation. ``app`` gets the public interface requirements by linking the target.

Expected behavior or output

``app`` can include headers from ``include`` but should not automatically see private source headers or the ``CORE_BUILD`` definition.

Concrete experiment

Change ``CORE_BUILD`` from ``PRIVATE`` to ``PUBLIC`` and print it from the application. Use the result to explain how an accidental public compile definition leaks implementation policy into consumers.

Worked example — Configure and test a standalone target graph

Trace the implementation

The optional directory contains a separate CMake project, not a file to paste over the root build. From the package root, configure it with `cmake -S source/book05/ch02/optional -B build/calibration`, build that directory, and run `ctest --test-dir build/calibration --output-on-failure`. Use the appropriate `-config` option with a multi-configuration generator.

`calibration` publishes its language level and header directory. The test receives them through `target_link_libraries` and adds only the private test-helper directory itself. No global `include_directories` command is needed. The default series build tests the same implementation independently of this standalone project.

The arithmetic fixture rejects nonfinite inputs and a nonfinite result. Exact equality is appropriate for the selected exactly representable values; it is not a universal rule for floating-point tests. A calibration product involving arbitrary decimal inputs should have a stated numerical tolerance and domain contract.

Complete worked source

The files below are in `source/book05/ch02/`. The source in this chapter and the companion are exported from the same authored data.

`calibration.hpp`

```
#ifndef HARBOR_CALIBRATION_HPP
#define HARBOR_CALIBRATION_HPP
namespace harbor { double calibrate(double raw, double gain, double offset); }
#endif
```

`calibration.cpp`

```
#include "calibration.hpp"
#include <cmath>
#include <stdexcept>
namespace harbor {
double calibrate(double raw, double gain, double offset) {
    if (!std::isfinite(raw) || !std::isfinite(gain) || !std::isfinite(offset))
        throw std::invalid_argument("nonfinite calibration input");
    const double result = raw * gain + offset;
    if (!std::isfinite(result)) throw std::overflow_error("calibration overflow");
    return result;
}
}
```

`main.cpp`

```
#include "check.hpp"
#include "calibration.hpp"
#include <limits>
#include <stdexcept>
```

```
int main() {
    CHECK(harbor::calibrate(4.0, 2.0, -1.0) == 7.0);
    bool rejected = false;
    try { (void)harbor::calibrate(std::numeric_limits<double>::infinity(), 1,
0); }
    catch (const std::invalid_argument&) { rejected = true; }
    CHECK(rejected);
    std::cout << "PASS\n";
}
```

Optional source: optional/CMakeLists.txt. This file has a separately stated build or execution procedure and is not part of the default build.

```
cmake_minimum_required(VERSION 3.24)
project(HarborCalibration LANGUAGES CXX)
include(CTest)
add_library(calibration STATIC ../calibration.cpp)
target_compile_features(calibration PUBLIC cxx_std_20)
target_include_directories(calibration PUBLIC "${CMAKE_CURRENT_LIST_DIR}/..")
set_target_properties(calibration PROPERTIES CXX_EXTENSIONS OFF)
add_executable(calibration_test ../main.cpp)
target_include_directories(calibration_test PRIVATE "$
{CMAKE_CURRENT_LIST_DIR}/../../support")
target_link_libraries(calibration_test PRIVATE calibration)
if(MSVC)
    target_compile_options(calibration PRIVATE /W4)
else()
    target_compile_options(calibration PRIVATE -Wall -Wextra -Wpedantic)
endif()
add_test(NAME calibration_contract COMMAND calibration_test)
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 2
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a second consumer without adding the library's include directory manually. Verify that the public requirement propagates. Then move a private implementation-only include into a private directory and confirm that a consumer cannot rely on it accidentally.

Export `compile_commands.json` with a supported generator and inspect the actual standard and include flags used for each translation unit.

Solution and review notes

The consumer should obtain public usage requirements solely by linking the library target. A private include path remains absent from that consumer's

command. This is a concrete test of the intended dependency boundary, not merely a visual check of CMake syntax.

The compilation database records per-translation-unit commands for tools such as clang-tidy. It does not replace link information or a full reproducibility manifest. Preserve the selected compiler, generator, cache options, and dependency identities alongside it when diagnosing build differences.

Technical references: [CMAKE](#) [CMAKETARGET](#) [NUMERIC](#)

Chapter 3 — Understanding Static and Dynamic Linking

Compilation produces object code with references that linking must resolve. Static and shared libraries arrange that resolution differently, which changes deployment, update behavior, visibility, and compatibility obligations. Harbor’s scale operation is built both ways in the optional experiment so the executable contract stays constant while the packaging changes.

3.1 Building Static Libraries

3.1.1 Archiving Object Files

A static library is commonly an archive of object files. Creating the archive groups compiled implementation units but does not generally resolve every external reference as a final executable link would. The eventual consumer still needs required dependencies and a compatible object format and ABI.

An archive is not automatically a standalone executable, and static linkage is not the same as compiling the entire operating system into a binary. Some system or runtime dependencies may remain dynamic. Inspect the actual artifact and final link command instead of inferring deployment properties from one library target’s `STATIC` keyword.

3.1.2 Linking Static Dependencies

The final linker selects code needed to satisfy references under the platform’s archive rules. Link order and cyclic dependencies can matter on some toolchains. Target-based dependencies make the intended relationships clearer, but they do not repair an incoherent circular architecture by themselves.

A change to statically linked library code normally requires relinking and redistributing affected consumers to deliver that change. Track those consumers in release planning. A security fix present in an updated archive on the build host does not modify already-shipped executables containing older object code.

Creating and Inspecting a Static Archive

Purpose

A static library is an archive of object files from which the linker extracts the members needed to satisfy unresolved symbols.

Code or command

```
g++ -std=c++20 -c src/add.cpp -Iinclude -o add.o
ar rcs libcalc.a add.o
ar t libcalc.a
g++ -std=c++20 app/main.cpp -Iinclude libcalc.a -o calculator
```

What the important parts do

``c`` compiles without linking. ``ar rcs`` creates an indexed archive, ``ar t`` shows its object members, and the final compiler-driver command invokes the linker with ``libcalc.a`` as an input.

Expected behavior or output

``ar t`` lists ``add.o``; the linked calculator runs without requiring ``libcalc.a`` at runtime because the selected object code is copied into the executable.

Concrete experiment

Add a second object file containing an unused function. Link again and inspect symbols or map output to see whether the linker extracts that archive member when no symbol requires it.

3.2 Building Shared Libraries

3.2.1 Understanding Dynamic Loading

A shared library is loaded or mapped as part of a process's runtime environment, and symbols are resolved according to the platform's loader rules. The executable must be able to locate a compatible library at runtime, not just while building. Search paths, runtime dependencies, and trusted installation locations are therefore part of deployment.

Replacing a shared object is safe only within its compatibility contract. Existing processes may continue using mappings of an older artifact, and a new file with the same name can still break consumers if its ABI changes. Record identity and compatibility rather than treating a filename as proof of interchangeability.

3.2.2 Managing Symbol Visibility

Export only the intended binary interface where the platform supports visibility controls. Hidden implementation symbols reduce accidental coupling and collisions, while Windows DLLs commonly use explicit import/export declarations. A public C++ header and a binary export list describe related but different interfaces.

The example's macro marks one function for export when building the shared variant and for import on Windows consumers. The default static build leaves the annotation empty. This controls visibility; it does not make C++ name mangling, exception handling, standard-library types, or compiler ABIs universally compatible.

Building a Shared Library with a Narrow Exported API

Purpose

Shared libraries remain separate runtime dependencies, so position-independent code and symbol visibility become part of the binary interface.

Code or command

```
g++ -std=c++20 -fPIC -fvisibility=hidden -c src/add.cpp -Iinclude  
-o add.o  
g++ -shared add.o -o libcalc.so  
nm -D --defined-only libcalc.so
```

What the important parts do

`-fPIC` produces code suitable for relocation in a shared object. `-fvisibility=hidden` makes symbols non-exported by default; a production library would explicitly mark only supported API symbols for export. `nm -D` inspects the dynamic symbol table.

Expected behavior or output

The library file is created. Depending on export attributes in `add.cpp`, the dynamic symbol list should contain only intentionally visible API symbols.

Concrete experiment

Build once without `-fvisibility=hidden` and compare `nm -D` output. Count how many implementation symbols become externally visible.

Worked example — Build the same contract as static and shared libraries

Trace the implementation

Configure the optional project in its own build/linkage directory, then build and run both registered tests. The root series build uses the static implementation; the optional project proves that the same source-level client can target either linkage choice when all components are built together with a compatible toolchain.

On Linux, inspect these trusted, locally built artifacts with `nm` and `readelf`. For example, `nm -D` on the shared library reveals dynamic symbols, while `readelf -d` on the shared consumer shows dynamic dependencies. Exact filenames and decorated symbol names are platform-specific. Do not run loader-based inspection commands on an untrusted downloaded executable merely as a documentation exercise.

The checked multiplication avoids relying on signed overflow. Its bounded domain is independent of linkage. This matters: replacing a static build with a shared build should not silently alter the API's numerical or error behavior. An ABI experiment needs an unchanged behavioral contract to be interpretable.

Complete worked source

The files below are in `source/book05/ch03/`. The source in this chapter and the companion are exported from the same authored data.

`scale.hpp`

```
#ifndef HARBOR_SCALE_HPP
#define HARBOR_SCALE_HPP
#if defined( WIN32 ) && defined( HARBOR_SHARED )
# if defined( HARBOR_BUILDING )
#  define HARBOR_API __declspec( dllexport )
# else
#  define HARBOR_API __declspec( dllimport )
# endif
#elif defined( __GNUC__ ) && defined( HARBOR_SHARED )
# define HARBOR_API __attribute__( (visibility("default")) )
#else
# define HARBOR_API
#endif
namespace harbor { HARBOR_API int scale( int value, int factor ); }
#endif
```

`scale.cpp`

```
#include "scale.hpp"
#include <limits>
#include <stdexcept>
namespace harbor {
int scale( int value, int factor ) {
    if ( value < -10000 || value > 10000 || factor < -10000 || factor > 10000 )
        throw std::invalid_argument( "scale domain" );
    const long long result = static_cast<long long>( value ) * factor;
    if ( result < std::numeric_limits<int>::min() || result >
```

```
std::numeric_limits<int>::max())
    throw std::overflow_error("int result");
    return static_cast<int>(result);
}
}
```

main.cpp

```
#include "check.hpp"
#include "scale.hpp"
int main() {
    CHECK(harbor::scale(6, 7) == 42);
    CHECK(harbor::scale(-3, 4) == -12);
    CHECK(harbor::scale(0, 10000) == 0);
    std::cout << "PASS\n";
}
```

Optional source: optional/CMakeLists.txt. This file has a separately stated build or execution procedure and is not part of the default build.

```
cmake_minimum_required(VERSION 3.24)
project(HarborLinkage LANGUAGES CXX)
include(CTest)
add_library(scale_static STATIC ../scale.cpp)
add_library(scale_shared SHARED ../scale.cpp)
foreach(target IN ITEMS scale_static scale_shared)
    target_compile_features(${target} PUBLIC cxx_std_20)
    target_include_directories(${target} PUBLIC "${CMAKE_CURRENT_LIST_DIR}/..")
endforeach()
target_compile_definitions(scale_shared PUBLIC HARBOR_SHARED PRIVATE
HARBOR_BUILDING)
set_target_properties(scale_shared PROPERTIES CXX_VISIBILITY_PRESET hidden
VISIBILITY_INLINES_HIDDEN YES)
foreach(kind IN ITEMS static shared)
    add_executable(test_${kind} ../main.cpp)
    target_include_directories(test_${kind} PRIVATE "$
{CMAKE_CURRENT_LIST_DIR}/../../support")
    target_link_libraries(test_${kind} PRIVATE scale_${kind})
    add_test(NAME ${kind}_contract COMMAND test_${kind})
endforeach()
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 3
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Compare exported symbols with visibility enabled and disabled in separate build directories. Identify the deliberately public scale symbol without assuming its mangled spelling is portable.

Change the library's public function signature, rebuild only the shared library, and analyze why an old consumer cannot be presumed compatible. Keep this experiment inside disposable build directories.

Solution and review notes

Visibility can reduce the dynamic symbol surface but cannot preserve a signature that has changed. A rebuilt consumer can adapt to a new header; an old executable contains the old binary expectations. Compatibility requires a deliberate versioning policy, not merely a successful rebuild of the library.

Record both the library artifact and consumer artifact in the experiment. If both are rebuilt, a passing test demonstrates a new coordinated build, not backward ABI compatibility. Chapter 20 returns to this distinction with a Pimpl boundary.

Technical references: [CMAKE](#) [CMAKETARGET](#) [ABI](#)

Chapter 4 — Managing Third-Party Dependencies

A dependency is code and a maintenance obligation. Before adding one, identify the capability needed, supported platforms, license terms, update process, and security ownership. The worked example uses a tiny original vendored helper so the integrity workflow can run offline; it is a model for dependency bookkeeping, not an endorsement of an external package.

4.1 Choosing Dependency Strategies

4.1.1 Vendoring Source Dependencies

Vendoring places a reviewed dependency snapshot inside the repository or distribution. It can make offline builds predictable and allow controlled patches, but it also makes your project responsible for tracking upstream fixes and preserving license and attribution materials. Copying a header is not the end of dependency management.

Keep local modifications distinguishable from upstream content. Record the origin, revision, and reason for each patch. A future update should make it possible to determine whether the patch is still needed rather than repeatedly copying a modified folder whose history nobody can reconstruct.

4.1.2 Using Package Managers

A package manager can resolve, retrieve, and build dependencies, often with platform profiles and binary caches. It reduces repetitive integration work but introduces its own configuration and trust inputs. Compiler ABI, runtime library choice, feature options, and architecture may affect whether a cached binary is compatible.

Prefer explicit manifests and lock information over a command that installs whatever is newest at build time. Distinguish a version constraint from a resolved version. Review transitive dependencies as well as direct ones, and design an offline or controlled-mirror strategy when network access is unavailable or restricted.

Pinning a Third-Party Source Dependency with FetchContent

Purpose

Dependency strategy becomes concrete when the build states where a dependency comes from, which version is selected, and which target consumes it.

Code or command

```
include(FetchContent)
FetchContent_Declare(
    fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG 11.0.2
)
FetchContent_MakeAvailable(fmt)
target_link_libraries(app PRIVATE fmt::fmt)
```

What the important parts do

`FetchContent\_Declare` records the source and immutable version choice; `GIT\_TAG` is the key reproducibility control here. The application links the dependency through its exported CMake target rather than guessing include or library paths.

Expected behavior or output

On a network-enabled build host, configuration retrieves the pinned revision and makes `fmt::fmt` available as a target.

Concrete experiment

Replace the tag with an intentionally nonexistent revision and observe configuration failure. Then compare this source-fetch approach with resolving the same package from a package manager lockfile.

4.2 Making Builds Reproducible

4.2.1 Pinning Dependency Versions

A pin identifies the exact input intended for a build. A release name is useful context; an immutable revision or a verified archive digest is stronger evidence of exact bytes. Mutable branch names alone do not define a reproducible source set.

A digest detects a mismatch against the recorded expectation, but it does not prove that the expected code is safe or that the expectation came from a trusted author. Verification and trust are separate. Update the pin through review with release notes, compatibility tests, and any required license changes rather than editing a checksum simply to make a failing check green.

4.2.2 Isolating Build Environments

Source identity is only one input. Compilers, standard libraries, linkers, build options, locale, environment variables, and generated metadata can affect the artifact. Record those inputs and use a clean build environment to expose accidental dependencies on a developer's machine.

A container image can standardize many inputs, but a mutable image tag is not an immutable environment and a container does not reproduce every host-kernel or hardware property. State the reproducibility goal: rebuilding equivalent behavior, reproducing a supported package, and producing identical bytes are increasingly strict claims with different evidence requirements.

Recording Toolchain and Dependency Inputs for a Reproducible Build

Purpose

A reproducible-build claim needs recorded inputs and artifact identity rather than only a successful compile.

Code or command

```
cmake --version
g++ --version | head -1
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --verbose > build-command.log 2>&1
sha256sum build/app > artifact.sha256
```

What the important parts do

The first two commands capture tool versions. Verbose build output preserves actual compile/link invocations. `sha256sum` records the exact resulting artifact so two build environments can compare byte identity.

Expected behavior or output

The process produces version evidence, a build-command log, and an artifact hash.

Concrete experiment

Build the same revision in two clean directories. Compare hashes first, then investigate timestamps, paths, generated metadata, and dependency/tool versions if they differ.

Worked example — Verify an offline vendored helper

Trace the implementation

The export contains `dependencies.json` with the SHA-256 digest of the supplied helper's actual bytes. Run `python3 source/book05/ch04/optional/verify_vendor.py` from the package root. The verifier resolves the path inside the chapter, reads the file as bytes, and compares its digest with the recorded value. It never downloads code or modifies the expected digest.

Change a copy of the vendored file and run the verifier against that copied chapter to observe a mismatch. Revert the copy from the known source rather than changing the manifest to match unreviewed content. The C++ test separately checks behavior; a file can have the expected digest and still contain a design defect.

This local manifest is intentionally not presented as a `vcpkg` or Conan lockfile. Those systems have their own schemas and resolution rules. The general lesson is to retain exact inputs and a reviewable update path, while using each actual package manager's documented format when adopting it.

Complete worked source

The files below are in `source/book05/ch04/`. The source in this chapter and the companion are exported from the same authored data.

`vendor/bounded.hpp`

```
#ifndef HARBOR_VENDOR_BOUNDED_HPP
#define HARBOR_VENDOR_BOUNDED_HPP
#include <stdexcept>
namespace harbor_vendor {
inline constexpr const char* version = "1.0.0-teaching";
inline int bounded(int value, int low, int high) {
    if (low > high) throw std::invalid_argument("reversed bounds");
    return value < low ? low : (value > high ? high : value);
}
}
#endif
```

`main.cpp`

```
#include "check.hpp"
#include "vendor/bounded.hpp"
#include <string_view>
int main() {
    CHECK(harbor_vendor::bounded(-2, 0, 10) == 0);
    CHECK(harbor_vendor::bounded(5, 0, 10) == 5);
    CHECK(harbor_vendor::bounded(30, 0, 10) == 10);
    CHECK(std::string_view(harbor_vendor::version) == "1.0.0-teaching");
    std::cout << "PASS\n";
}
```

Optional source: `optional/verify_vendor.py`. This file has a separately stated build or execution procedure and is not part of the default build.

```
#!/usr/bin/env python3
"""Check the exact vendored bytes against the reviewed local manifest."""
from pathlib import Path
import hashlib
import json
import sys
root = Path(__file__).resolve().parents[1]
manifest = json.loads((root / "dependencies.json").read_text(encoding="utf-8"))
for item in manifest["dependencies"]:
    path = (root / item["path"]).resolve()
    if not path.is_relative_to(root):
        raise SystemExit("dependency path escapes chapter")
    actual = hashlib.sha256(path.read_bytes()).hexdigest()
    if actual != item["sha256"]:
        raise SystemExit(f"integrity mismatch: {item['path']}")
    print(f"verified {item['name']} {item['version']}")
sys.exit(0)
```

vendor/ORIGIN.txt

Original teaching helper authored for this series; not copied from a third-party project.
Version: 1.0.0-teaching. Preserve this origin record when adapting the integrity exercise.
The manifest demonstrates byte identity, not an external signature or a security audit.

dependencies.json

```
{
  "dependencies": [
    {
      "name": "harbor-bounded",
      "version": "1.0.0-teaching",
      "origin": "Original source supplied with this series",
      "path": "vendor/bounded.hpp",
      "sha256":
"7a2d754b4440deb2fe8833022287bd550f08917f77303a35497a2e08c1946d43"
    }
  ]
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 4
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a second vendored file and extend the manifest with its real digest. Modify only whitespace and explain why a byte-level integrity check still fails even when the C++ behavior is unchanged.

Write a dependency-update checklist covering origin, license, direct and transitive versions, local patches, supported platforms, and regression evidence.

Solution and review notes

SHA-256 measures byte identity, so formatting changes alter the digest. That is appropriate when the claim is exact source identity. Behavioral equivalence is a different question answered by review and tests; do not weaken the integrity check to approximate it.

An update is complete only when the recorded input and the tested build agree. Retain the old pin and evidence so a regression can be reproduced. A rollback plan should identify the complete compatible dependency set rather than one isolated library file.

Technical references: [CMAKE DEPENDENCIES](#)

Chapter 5 — Managing C++ Development with Git

Version control preserves a reviewable history of decisions, not just a backup of the latest folder. A useful C++ change includes its implementation, tests, and build implications together. This chapter uses a bounded parser as a sample change and an optional temporary-repository exercise to demonstrate branching without touching any existing repository.

5.1 Organizing Changes into Commits

5.1.1 Creating Focused Commits

A focused commit has one coherent purpose that a reviewer can explain. It may touch several files when the implementation, declaration, test, and build rule belong to the same behavior change. Conversely, unrelated formatting and feature work can obscure each other even when they fit in one file.

Inspect the staged diff before committing. Git's working tree, index, and committed snapshot are distinct states; testing unstaged changes does not prove the staged snapshot passes. A clean checkout or worktree can verify exactly the candidate revision. Do not commit credentials, large build directories, or generated artifacts merely because a broad add command selected them.

5.1.2 Writing Useful Commit Messages

A useful subject names the change in behavior, and the body explains why the prior behavior was inadequate and what constraints shaped the solution. The diff already shows syntax. Preserve the rejected alternatives or compatibility consequences that a future maintainer cannot recover from code alone.

For a parsing fix, describe the accepted grammar and the formerly mishandled case. Include test evidence that refers to the actual revision and configuration. Avoid declaring an issue fully solved when the change covers only one path; precise messages make later bisect and regression analysis more effective.

Creating One Focused Git Commit from a Single Logical Change

Purpose

A focused commit should contain the implementation and tests for one logical change, with staged content inspected before it becomes history.

Code or command

```
git status --short
git diff -- src/parser.cpp tests/parser_test.cpp
git add src/parser.cpp tests/parser_test.cpp
git diff --cached
git commit -m "Reject truncated parser frames"
```

What the important parts do

`git diff` reviews working-tree changes, `git add` selects only the relevant files, and `git diff --cached` verifies the exact staged patch. The message states the behavioral change rather than a vague activity such as “updates.”

Expected behavior or output

The resulting commit contains only the parser fix and its test if the staged diff was clean.

Concrete experiment

Make an unrelated README edit before staging. Practice leaving it unstaged while committing only the parser change, then verify with `git show --stat HEAD`.

5.2 Collaborating Through Branches and Reviews

5.2.1 Managing Feature Branches

A branch names a movable line of development. Keep a feature branch small enough to review, integrate from the agreed base, and use the team’s policy for merge or rebase. Rewriting a branch that others already consume can disrupt their work, so coordinate rather than treating history edits as purely local cleanup.

Reviews should examine interfaces, ownership, failure paths, tests, and compatibility, not only style. A passing automated check is one input to that review. It cannot determine whether a changed contract is appropriate for consumers whose expectations were never encoded in the suite.

5.2.2 Resolving Merge Conflicts

A textual conflict marks an area Git cannot combine automatically. Resolve the intended combined behavior, remove conflict markers, and run relevant tests. Choosing one side wholesale may discard a needed semantic change even when the resulting file compiles.

A clean automatic merge can also be semantically wrong. Two branches may change different files while violating a shared invariant. Integration tests and interface review therefore remain necessary after conflict-free merges. Preserve a recoverable working state before experimenting with history commands, and perform tutorial conflict exercises only in disposable repositories.

Resolving a Feature-Branch Merge Conflict with Evidence

Purpose

Branch collaboration becomes an engineering task when two histories touch the same lines and the merged behavior must be reviewed rather than mechanically accepted.

Code or command

```
git switch -c feature/parser-limit
# edit and commit the feature
git switch main
git merge feature/parser-limit
# after resolving conflicts:
git add src/parser.cpp
git diff --cached
git commit
```

What the important parts do

`git merge` combines histories and can stop on conflicts. The developer edits the conflicted file into the intended final state, stages the resolution, and inspects the staged result before recording the merge commit.

Expected behavior or output

After a correct resolution, `git status` is clean and tests should pass on the merged tree.

Concrete experiment

Create two branches that change the same validation condition differently. Resolve the conflict, then run the affected tests before committing so the resolution is behaviorally verified.

Worked example — Review a parser change and rehearse a branch safely

Trace the implementation

The C++ parser requires complete consumption of the input token. That makes 12x invalid rather than accepting the leading 12 and silently ignoring the remainder. A focused commit for this behavior would include the parser and the trailing-garbage regression case together.

Run the optional Python script to rehearse two commits, a feature branch, and a fast-forward integration. It configures identity only inside a newly created temporary repository and removes that repository on exit. It does not set global Git configuration, connect to a remote, or mutate your current project. The printed commit identifiers are actual results of that run and are not expected to match across machines or times.

The exercise deliberately uses a conflict-free fast-forward before introducing conflicts in the lab. It distinguishes learning the mechanics from changing a real team's history. An existing repository should be inspected with status and diff before any consequential operation.

Complete worked source

The files below are in `source/book05/ch05/`. The source in this chapter and the companion are exported from the same authored data.

main.cpp

```
#include "check.hpp"
#include <charconv>
#include <optional>
#include <string_view>
std::optional<unsigned> parse_count(std::string_view text) {
    unsigned value{};
    const auto [end, error] = std::from_chars(text.data(), text.data()
+text.size(), value);
    if (error != std::errc{} || end != text.data()+text.size() || value > 1000)
return std::nullopt;
    return value;
}
int main() {
    CHECK(parse_count("0") == 0);
    CHECK(parse_count("1000") == 1000);
    CHECK(!parse_count("1001") && !parse_count("12x") && !parse_count("-1"));
    std::cout << "PASS\n";
}
```

Optional source: `optional/git_rehearsal.py`. This file has a separately stated build or execution procedure and is not part of the default build.

```
#!/usr/bin/env python3
"""Create and remove a temporary repository; never alters the caller's
repository."""
from pathlib import Path
import shutil
```

```

import subprocess
import tempfile
if not shutil.which("git"):
    raise SystemExit("git is required for this optional exercise")
with tempfile.TemporaryDirectory(prefix="harbor-git-") as directory:
    root = Path(directory)
    def git(*arguments):
        return subprocess.run(["git", *arguments], cwd=root, check=True,
                               text=True, capture_output=True).stdout.strip()
    git("init", "--initial-branch=main")
    git("config", "user.name", "Harbor Teaching Fixture")
    git("config", "user.email", "fixture@example.invalid")
    (root / "contract.txt").write_text("count: 0..1000\n", encoding="utf-8")
    git("add", "contract.txt")
    git("commit", "-m", "Define bounded count contract")
    git("switch", "-c", "test-boundaries")
    (root / "tests.txt").write_text("accept 0 and 1000; reject 1001\n",
    encoding="utf-8")
    git("add", "tests.txt")
    git("commit", "-m", "Document count boundary cases")
    git("switch", "main")
    git("merge", "--ff-only", "test-boundaries")
    if git("status", "--porcelain"):
        raise SystemExit("unexpected dirty working tree")
    print(git("log", "--oneline", "-2"))
    print("PASS: temporary repository integrated with a clean tree")

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 5
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

In a separate temporary repository, create two branches that change the same contract line differently. Attempt a merge, inspect the conflict, and resolve it by writing the intended combined requirement. Add a test that would catch choosing the wrong side.

Write a commit message for rejecting trailing characters in `parse_count`. Explain the behavior change rather than saying only that parsing was improved.

Solution and review notes

A useful message identifies complete-token parsing and the 12x regression. The test protects the new contract. During the conflict exercise, the correct resolution follows the desired grammar and range, not which branch is newer.

After resolving, stage the resolved file, complete the merge according to Git's instructions, and run the combined tests from the resulting revision. An empty

status output proves the working tree is clean, not that the program is correct.
Both observations belong in the integration record.

Technical references: [GIT](#) [CHARCONV](#)

Chapter 6 — Designing Effective Unit Tests

A unit test states a behavioral claim about a small component under controlled conditions. Harbor’s count parser accepts a precise decimal grammar and range; testing it means identifying both the permitted values and the representations that must be rejected. The goal is not to maximize the number of assertions indiscriminately, but to make failures local, reproducible, and informative.

6.1 Isolating Small Units of Behavior

6.1.1 Structuring Test Cases

Arrange the input and dependencies, perform one meaningful operation, and examine the result. A case should make its purpose visible: minimum valid count, maximum valid count, empty text, overflow, or trailing garbage. Several assertions can belong to one case when they describe the same outcome, but an unrelated cascade obscures the first broken contract.

Keep setup independent of the behavior being tested. Constructing the expected answer with the same parser implementation merely repeats the potential defect. Prefer explicit small examples, an independently reasoned oracle, or a separately implemented reference with different failure modes.

6.1.2 Defining Expected Outcomes

Specify success values and failure categories. A test that only checks that something threw may accept the wrong exception from a setup bug. Likewise, a nonempty output string is weaker evidence than the exact expected record when the format is part of the API.

Use comparisons appropriate to the domain. Exact equality is suitable for bounded integer parsing; floating-point computation may require an absolute and relative error contract. Keep test checks active in release configurations. The series’ CHECK helper throws with a source location instead of disappearing when NDEBUG is defined.

Testing One Pure Function with Arrange-Act-Assert Structure

Purpose

A small unit test is strongest when it isolates one behavior, supplies a controlled input, and states the expected result directly.

Code or command

```
#include <cassert>

int clamp_percent(int x){ return x<0?0:(x>100?100:x); }

int main(){
    const int input = 120;           // arrange
    const int actual = clamp_percent(input); // act
    assert(actual == 100);           // assert
}
```

What the important parts do

The function has no global state or I/O, so the test can exercise it deterministically. The arrange/act/assert comments make the test's three responsibilities explicit.

Expected behavior or output

The program exits normally when `clamp_percent(120)` returns 100; a failed assertion terminates the test.

Concrete experiment

Add independent tests for -1, 0, 50, 100, and 101. Keep each failure attributable to one boundary rule.

6.2 Testing Edge and Failure Cases

6.2.1 Testing Boundary Conditions

Boundaries divide behavior classes. For a range from zero through 65,535, test both endpoints, neighbors, and representations near conversion overflow. Empty and single-character inputs exercise paths that large ordinary values can miss. A fixed maximum length is another boundary independent of the numeric maximum.

Property-style tests complement examples. Formatting an accepted integer and parsing it should recover the same value under the specified grammar. Such a round trip does not prove rejection behavior, so retain explicit malformed-input cases instead of treating one property as a complete specification.

6.2.2 Testing Error Paths

A failure path is part of the public contract. Verify that rejected input does not leave partially mutated state, leak an owned resource, or return a success-like sentinel that callers cannot distinguish from a legitimate value. Prefer explicit error results when failure is routine data validation.

Fault injection should target a defined boundary, such as an I/O adapter returning failure or an allocator refusing a request. Randomly making arbitrary operations fail can create noise without demonstrating the desired guarantee. Record which failure point was exercised and what invariant remained true afterward.

Exercising Boundary Values and an Explicit Parse Failure Path

Purpose

Boundary testing should target values immediately inside and outside the accepted domain as well as malformed input that drives a separate error path.

Code or command

```
#include <charconv>
#include <optional>
#include <string_view>
#include <cassert>

std::optional<int> parse_port(std::string_view s){
    int v{}; auto [p,ec]=std::from_chars(s.data(),s.data()+s.size(),v);
    if(ec!=std::errc{} || p!=s.data()+s.size() || v<1 || v>65535)
        return std::nullopt;
    return v;
}

int main()
{assert(parse_port("1"));assert(parse_port("65535"));assert(!
parse_port("0"));assert(!parse_port("65536"));assert(!
parse_port("abc"));}

```

What the important parts do

`from_chars` reports conversion status without exceptions. The function rejects partial parses, ports below 1, and ports above 65535. The assertions cover both valid endpoints and invalid neighbors.

Expected behavior or output

All assertions pass for the stated port contract.

Concrete experiment

Add an empty string and `"80x"`. Predict which validation condition rejects each input before running the tests.

Worked example — Build a table-driven parser test

Trace the implementation

The parser's grammar explicitly permits leading zeroes within five characters and rejects signs and whitespace. Those decisions are visible in the table rather than inherited accidentally from a conversion function. The range includes zero because this is a numeric port-field parser; a connection policy may separately prohibit zero for a remote destination.

The implementation checks character membership before conversion and still verifies `from_chars`'s error and end pointer. The table tests the numeric maximum, a value just above it, and a six-character representation whose numeric value would otherwise be small. The sampled round-trip property covers many accepted values without depending on random input or nondeterministic seeds.

The test does not claim exhaustive proof of every byte sequence. It provides explicit behavioral examples plus a property over a selected finite set. An exhaustive test over all accepted integer values is inexpensive here, but rejection still requires its own strategy because the set of possible strings is much larger.

Complete worked source

The files below are in `source/book05/ch06/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <array>
#include <charconv>
#include <optional>
#include <string>
#include <string_view>

std::optional<unsigned> parse_port(std::string_view text) {
    if (text.empty() || text.size() > 5) return std::nullopt;
    for (char character : text)
        if (character < '0' || character > '9') return std::nullopt;
    unsigned value{};
    const auto [end, error] = std::from_chars(text.data(), text.data()
+text.size(), value);
    if (error != std::errc{} || end != text.data()+text.size() || value > 65535)
        return std::nullopt;
    return value;
}

int main() {
    struct Case { std::string_view text; std::optional<unsigned> expected; };
    const std::array cases{
        Case{"0", 0}, Case{"1", 1}, Case{"65535", 65535}, Case{"00001", 1},
        Case{"65536", std::nullopt}, Case{"", std::nullopt}, Case{"-1",
std::nullopt},
        Case{"+1", std::nullopt}, Case{"1x", std::nullopt}, Case{" 1",
std::nullopt},
        Case{"000001", std::nullopt}
    };
    for (const auto& item : cases) CHECK(parse_port(item.text) ==
item.expected);
    for (unsigned value = 0; value <= 65535; value += 31)
        CHECK(parse_port(std::to_string(value)) == value);
    std::cout << "table_cases=" << cases.size() << "\nPASS\n";
}
```

Run and verify

From the extracted package root:

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Change the round-trip loop to cover every accepted integer and measure the test's runtime. Add embedded NUL and non-ASCII byte cases using `string_view` with an explicit length.

Define a different policy that forbids leading zeroes except for the single character zero. Update the tests first, then implement the policy without changing the numeric conversion contract accidentally.

Solution and review notes

An embedded NUL is not the end of a `string_view` when its length includes that byte; the character check rejects it. This catches assumptions inherited from null-terminated C strings. Non-ASCII bytes are rejected by direct comparisons without passing a negative char to a ctype function.

The no-leading-zero policy adds a representation rule before conversion: a multi-character token beginning with zero is invalid. Keep this policy separate from the accepted numeric range so the resulting tests explain which requirement caused rejection.

Technical references: [CHARCONV GUIDELINES](#)

Chapter 7 — Designing Integration and System Tests

A parser can pass every unit test while a file adapter writes the wrong format or forgets to report a failed stream. Integration tests exercise those boundaries together. This chapter persists a bounded sequence of measurements, reloads it through the public adapter, and verifies both successful composition and a malformed-record failure.

7.1 Testing Component Interactions

7.1.1 Verifying Interface Contracts

An interface contract includes representation, ownership, error behavior, and limits. A writer and reader must agree on delimiters and numeric grammar, not merely on the fact that both use text. A network sender and receiver similarly need framing and byte-order agreement beyond sharing the type string.

Test the boundary with independently constructed expected bytes. A round trip can conceal a paired bug when both sides make the same wrong assumption. Combine a golden representation test with round-trip tests and malformed-input cases to expose different classes of mismatch.

7.1.2 Testing Real Dependencies

Some behavior exists only with a real dependency: filesystem permissions, socket framing, database transaction boundaries, or loader search paths. Use a controlled real instance for integration tests where a fake cannot reproduce the relevant mechanism. Keep the instance isolated and explicitly cleaned up.

A real dependency makes the test more representative but can add timing, platform, and environmental variability. State which environment is required and classify a missing capability separately from a failed contract. Never silently substitute an in-memory fake while reporting that the real dependency was tested.

Verifying a Component Contract Through an In-Memory Repository

Purpose

A component test can verify an interface contract through a realistic but lightweight implementation, including both success and absence behavior.

Code or command

```
#include <map>
#include <optional>
#include <string>
#include <cassert>

struct Repo{virtual ~Repo()=default;virtual void
put(int,std::string)=0;virtual std::optional<std::string>
get(int)=0;};
struct MemoryRepo:Repo{std::map<int,std::string> m;void put(int
k,std::string v)override{m[k]=std::move(v);}
std::optional<std::string> get(int k)override{auto
i=m.find(k);if(i==m.end())return {};return i->second;}};
int main(){MemoryRepo
r;r.put(7,"seven");assert(r.get(7)=="seven");assert(!r.get(8));}
```

What the important parts do

`Repo` defines the contract, while `MemoryRepo` supplies actual storage semantics without external infrastructure. The test interacts only through the repository behavior it expects callers to rely on.

Expected behavior or output

The assertions confirm one stored value can be retrieved and a missing key returns no value.

Concrete experiment

Add a second repository implementation backed by a temporary file and run the same contract tests against both implementations.

7.2 Testing Complete Applications

7.2.1 Designing End-to-End Scenarios

An end-to-end scenario starts from a supported user entry point and checks an externally observable result. For a command-line program, that includes arguments, input files, standard output, standard error, exit status, and any created artifacts. Testing only a library function does not establish the CLI's behavior.

Choose a small number of representative workflows and preserve detailed unit coverage below them. Excessively broad scenarios can be slow and hard to diagnose. A failed complete workflow should produce enough evidence to locate the responsible layer rather than requiring a manual rerun that may lose the original state.

7.2.2 Capturing Reproducible Failures

Record the exact executable identity, arguments, fixture bytes, environment assumptions, and observed outputs. Use deterministic seeds and bounded inputs when generating cases. A screenshot of a generic error is weaker evidence than a minimal fixture that another engineer can run against the same build.

Do not leak secrets into retained test artifacts. Production-derived data may need redaction or replacement with a synthetic reproducer. Retain only the context needed to demonstrate the defect, and label whether the scenario used a real service, a local fixture, or a simulated failure.

End-to-End CLI Scenario with Captured Exit Code and Output

Purpose

An end-to-end test treats the built application as a user would: invoke the executable, provide a fixture, and validate externally observable behavior.

Code or command

```
set -euo pipefail
output=$(./app --input fixtures/valid.txt)
status=$?
printf 'status=%s output=%s\n' "$status" "$output"
test "$status" -eq 0
test "$output" = "records=3"
```

What the important parts do

Command substitution captures standard output, `\$?` captures the process status, and the final `test` commands turn the scenario into a reproducible pass/fail check.

Expected behavior or output

The scenario passes only when the application exits 0 and prints exactly `records=3` for the known fixture.

Concrete experiment

Create a malformed fixture and assert both a nonzero exit status and a stable diagnostic message. Keep the failing input in version control so the failure is reproducible.

Worked example — Round-trip a real measurement file

Trace the implementation

Run this chapter through the supplied fixture runner, which creates a fresh working directory for each case. The test intentionally creates `measurements.fixture` there. Do not invoke fixture-writing examples in a directory containing a real file with the same name. The final package's README distinguishes these fixture executables from tools intended to process user data.

The golden byte comparison checks the writer independently of the reader. The malformed-file case then verifies that loading does not return a partially accepted vector after encountering 2x. Token length and record count are bounded while reading, before unbounded accumulation can occur.

The save routine validates all values before opening the destination, reducing one class of destructive failure. It still truncates an existing file and is not an atomic replacement or crash-durable transaction. A production save operation may need a same-filesystem temporary file, explicit durability requirements, and platform-specific synchronization. Integration testing should state which of those guarantees is actually implemented.

Complete worked source

The files below are in `source/book05/ch07/`. The source in this chapter and the companion are exported from the same authored data.

store.hpp

```
#ifndef HARBOR_STORE_HPP
#define HARBOR_STORE_HPP
#include <filesystem>
#include <span>
#include <vector>
namespace harbor {
void save_values(const std::filesystem::path&, std::span<const int>);
std::vector<int> load_values(const std::filesystem::path&);
}
#endif
```

store.cpp

```
#include "store.hpp"
#include <charconv>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <string>
namespace harbor {
void save_values(const std::filesystem::path& path, std::span<const int> values)
{
    if (values.size() > 1024) throw std::invalid_argument("record count");
    std::ostringstream candidate;
    for (int value : values) {
        if (value < -1000000 || value > 1000000) throw
std::invalid_argument("record value");
    }
}
```

```

        candidate << value << '\n';
    }
    std::ofstream output(path, std::ios::binary | std::ios::trunc);
    if (!output) throw std::runtime_error("open output");
    output << candidate.str();
    output.flush();
    if (!output) throw std::runtime_error("write output");
    output.close();
    if (!output) throw std::runtime_error("close output");
}

std::vector<int> load_values(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) throw std::runtime_error("open input");
    std::vector<int> result;
    std::string token;
    auto finish = [&] {
        if (token.empty()) return;
        if (result.size() == 1024) throw std::runtime_error("record count");
        int value{};
        const auto [end, error] = std::from_chars(token.data(), token.data()
+token.size(), value);
        if (error != std::errc{} || end != token.data()+token.size() || value <
-1000000 || value > 1000000)
            throw std::runtime_error("record value");
        result.push_back(value); token.clear();
    };
    char character{};
    while (input.get(character)) {
        if (character == ' ' || character == '\n' || character == '\r' ||
character == '\t') finish();
        else {
            if (token.size() == 16) throw std::runtime_error("record length");
            token.push_back(character);
        }
    }
    if (!input.eof()) throw std::runtime_error("read input");
    finish();
    return result;
}
}

```

main.cpp

```

#include "check.hpp"
#include "store.hpp"
#include <filesystem>
#include <fstream>
#include <iterator>
#include <string>
#include <vector>
int main() {
    // The supplied fixture runner gives this test a fresh working directory.
    const auto path = std::filesystem::path{"measurements.fixture"};
    const std::vector values{3, -2, 8};
    harbor::save_values(path, values);
    CHECK(harbor::load_values(path) == values);
    std::ifstream input(path, std::ios::binary);
    const std::string bytes((std::istreambuf_iterator<char>(input)), {});
    CHECK(bytes == "3\n-2\n8\n");
    input.close();
    { std::ofstream bad(path, std::ios::trunc); bad << "3\n2x\n"; }
    bool rejected = false;
    try { (void)harbor::load_values(path); }
    catch (const std::runtime_error&) { rejected = true; }
    CHECK(rejected);
}

```

```
CHECK(std::filesystem::remove(path));  
std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 7
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a missing-file case and a token longer than sixteen bytes. Verify the error category and confirm that no partial result is returned. Then add a CLI wrapper and test its exit status and output separately from the library.

Design, but do not assume, an atomic-replacement contract. Explain how failure before replacement differs from failure after replacement but before durable storage.

Solution and review notes

A local result vector disappears during exception unwinding, so no partial result is published by this API. The caller receives either a complete accepted sequence or an error. A CLI test should also verify that errors go to standard error and return nonzero.

Atomic namespace replacement and crash durability are different properties. A rename can make readers see one complete file while still requiring additional synchronization for a defined power-loss guarantee. Specify the target platform and filesystem assumptions before claiming either property.

Technical references: [FILESYSTEM](#) [IO](#) [CHARCONV](#)

Chapter 8 — Isolating Dependencies with Test Doubles

A retry or cache policy often depends on time and external results. Waiting for real minutes or relying on an unreliable service makes unit tests slow and nondeterministic. Harbor’s short-lived cache accepts a clock and a source explicitly so tests can advance time and control failures without disguising those doubles as real infrastructure.

8.1 Understanding Stubs, Fakes, and Mocks

8.1.1 Replacing Expensive Dependencies

A stub returns configured answers, a fake supplies a simplified working implementation, and a mock commonly verifies expected interactions. The terms vary among frameworks, so document the role rather than arguing over a label. Choose the smallest double that exposes the contract under test.

A fake clock is appropriate for expiration logic because the policy needs an ordered time value, not actual sleeping. It is not evidence that a platform timer meets a deadline. Likewise, an in-memory repository can test application rules but cannot establish a database’s isolation or durability behavior.

8.1.2 Controlling Test Inputs

Make time, randomness, and external responses explicit inputs at the boundary where they affect decisions. A deterministic test can arrange a hit, expiration, or source failure precisely. Hidden calls to a global clock make such cases harder to reproduce and tempt tests to use fragile sleep intervals.

Keep the double’s behavior faithful to the interface contract. A fake that always succeeds can hide error paths; a mock that enforces incidental call order can make harmless refactoring break tests. Verify interactions only when they are an actual requirement, such as avoiding a costly refresh before expiration.

Replacing a Slow Clock Dependency with a Controllable Fake

Purpose

A fake replaces an expensive or uncontrollable dependency with a working in-memory implementation whose state the test can set precisely.

Code or command

```
#include <chrono>
#include <cassert>

struct Clock{virtual ~Clock()=default;virtual long long
now_ms()const=0;};
struct FakeClock:Clock{long long value{};long long now_ms()const
override{return value;}};

bool expired(const Clock& c,long long deadline){return
c.now_ms()>=deadline;}
int main(){FakeClock c;c.value=999;assert(!
expired(c,1000));c.value=1000;assert(expired(c,1000));}
```

What the important parts do

`Clock` is the dependency boundary. `FakeClock` returns a test-controlled value instead of reading wall time, so the test can exercise both sides of the deadline without sleeping.

Expected behavior or output

The first assertion sees “not expired”; the second sees expiration exactly at the boundary.

Concrete experiment

Add a counting fake that records how many times `now\_ms` is called. Use it to detect an implementation that redundantly queries the clock inside one operation.

8.2 Designing Code for Testability

8.2.1 Injecting Dependencies

Dependency injection means supplying a dependency rather than constructing or locating it secretly inside the component. Constructor references, callable objects, templates, or an explicit context can all implement it. A virtual interface is useful when runtime substitution is needed, but it is not mandatory for every small function.

The dependency's lifetime must exceed the borrower's use. An injected reference makes borrowing visible but does not extend the referred object's lifetime. Ownership and testability should reinforce each other: identify who creates the dependency, who may mutate it, and when the consumer stops using it.

8.2.2 Avoiding Hidden Global Dependencies

Global mutable state couples tests and makes order matter. A cache that shares one process-wide source or clock can pass alone and fail after another case changes that state. Isolated instances make each case's initial conditions explicit.

Some process-wide services are legitimate, but wrap their access behind a reviewed boundary and control reset or lifetime policy. Do not replace one hidden singleton with another test-only singleton. The objective is reproducibility and clear dependencies, not simply the presence of an interface class.

Injecting a Dependency Instead of Reading Hidden Global State

Purpose

Dependency injection makes test inputs visible in the object constructor rather than hiding behavior behind mutable globals or environment lookups.

Code or command

```
#include <string>
#include <cassert>

struct Config{std::string prefix;};
class Formatter{
    const Config& config_;
public:
    explicit Formatter(const Config& c):config_(c){}
    std::string format(std::string s)const{return config_.prefix+s;}
};
int main(){Config c{"test:"};Formatter
f(c);assert(f.format("ok")== "test:ok");}
```

What the important parts do

`Formatter` stores a reference to a supplied `Config`. The test owns that configuration and can choose deterministic values. There is no ambient singleton whose state can leak between tests.

Expected behavior or output

The assertion passes with `test:ok`.

Concrete experiment

Write a second test with prefix `prod:` in the same process. Compare this with a global configuration design that would require mutation and reset between tests.

Worked example — Test cache expiration without sleeping

Trace the implementation

The cache borrows the clock and source, both of which outlive it in main. A cached value is valid for elapsed ticks strictly less than ten. The tests inspect tick nine and tick ten separately, making the expiration boundary explicit without waiting in real time.

The refresh computes a candidate before committing cache state. If the source throws, the old value and timestamp remain intact, but get propagates the failure rather than silently returning stale data. That is one policy choice; a stale-on-error

policy would need a different contract and tests. A backward clock value forces refresh rather than subtracting into a misleading negative age.

The clock domain is bounded so subtraction is safe. The fake uses integer ticks rather than claiming to emulate a real `steady_clock` implementation. The source call count is asserted because avoiding unnecessary refreshes is part of this cache's behavior, not merely an incidental implementation detail.

Complete worked source

The files below are in `source/book05/ch08/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <cstdint>
#include <optional>
#include <stdexcept>

struct Clock { virtual ~Clock() = default; virtual std::int64_t now() const = 0; };
struct Source { virtual ~Source() = default; virtual int fetch() = 0; };
class Cache {
    Clock& clock_;
    Source& source_;
    std::optional<int> value_;
    std::int64_t stored_at_;
public:
    Cache(Clock& clock, Source& source) : clock_(clock), source_(source) {}
    int get() {
        const auto now = clock_.now();
        if (now < 0 || now > 1000000000) throw std::runtime_error("clock
domain");
        if (value_ && now >= stored_at_ && now - stored_at_ < 10) return
*value_;
        const int candidate = source_.fetch(); // Failure leaves prior state
unchanged.
        value_ = candidate; stored_at_ = now;
        return candidate;
    }
};
struct FakeClock final : Clock {
    std::int64_t tick{};
    std::int64_t now() const override { return tick; }
};
struct CountingSource final : Source {
    int calls{}, value{7}; bool fail{};
    int fetch() override { ++calls; if (fail) throw std::runtime_error("source
failure"); return value; }
};
int main() {
    FakeClock clock; CountingSource source; Cache cache{clock, source};
    CHECK(cache.get() == 7 && source.calls == 1);
    clock.tick = 9; source.value = 8;
    CHECK(cache.get() == 7 && source.calls == 1);
    clock.tick = 10;
    CHECK(cache.get() == 8 && source.calls == 2);
    clock.tick = 20; source.fail = true;
    bool failed = false;
    try { (void)cache.get(); } catch (const std::runtime_error&) { failed =
true; }
```

```
    CHECK(failed && source.calls == 3);
    source.fail = false; source.value = 9;
    CHECK(cache.get() == 9 && source.calls == 4);
    std::cout << "PASS\n";
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 8
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a backward-clock case and a source that fails on its first call. Verify that a failed initial fetch does not create a cache entry. Then define a stale-on-error policy with an explicit maximum stale age.

Replace the virtual interfaces with injected callables and compare the ownership and compilation consequences, keeping the same behavioral tests.

Solution and review notes

With no successful prior fetch, failure must leave `value_disengaged`. The next call should attempt the source again. A stale policy must distinguish usable stale data from a value too old to serve; do not silently turn unlimited staleness into availability.

Callable injection can remove virtual dispatch and improve inlining, but template-based consumers may expose more implementation in headers and generate more instantiations. Neither form is automatically superior. Choose based on substitution, build boundaries, and measured costs.

Technical references: [GUIDELINES](#) [LIFETIME](#) [CHRONO](#)

Chapter 9 — Debugging Native C++ Applications

Debugging is an evidence-gathering process. A debugger can show where a program stopped, what values it currently holds, and how execution reached that point, but those observations still require a hypothesis about the intended contract. This chapter supplies a small deterministic calculation with named stages and a debugger command file so the investigation can be repeated without an intentional crash.

9.1 Controlling Programs with GDB or LLDB

9.1.1 Setting Breakpoints and Watchpoints

A breakpoint stops execution at a function, source location, or condition. A watchpoint observes changes to a selected memory location, subject to debugger and hardware limits. Use them to test a hypothesis: where does this value first violate its bound, or which caller supplies an unexpected argument?

Optimized code can inline functions, remove variables, and reorder operations while preserving allowed behavior. Debug information helps map the artifact to source but cannot recreate every eliminated intermediate. Begin with a suitable debug build, then reproduce on the relevant optimized artifact when optimization changes the symptom.

9.1.2 Inspecting Variables and Stack Frames

A stack trace identifies active calls as reconstructed by the debugger. Select a frame to inspect its parameters and locals in that frame's context. A pointer's numeric value is only a starting point; establish the pointee's type, lifetime, and valid extent before treating memory as a meaningful object.

Keep the executable, symbols, and source revision aligned. A plausible source line from mismatched symbols can send an investigation in the wrong direction. Record compiler flags and any separate debug-symbol artifact as part of the failure evidence.

Stopping at a Fault-Relevant Line and Inspecting a Stack Frame in GDB

Purpose

Interactive debugging is about controlling execution and inspecting program state at the exact point where assumptions become false.

Code or command

```
g++ -std=c++20 -O0 -g crash.cpp -o crash
gdb ./crash
(gdb) break process_record
(gdb) run
(gdb) info locals
(gdb) print record_id
(gdb) backtrace
```

What the important parts do

`-g` retains source-level debug information and `-O0` keeps the exercise easy to correlate with source. The breakpoint stops before continuing through `process_record`; `info locals`, `print`, and `backtrace` reveal the current frame and call chain.

Expected behavior or output

GDB stops in `process_record` and displays the selected variable and stack frames according to the current test input.

Concrete experiment

Add a watchpoint on a variable that changes unexpectedly. Continue execution until GDB stops on the write that modified it.

9.2 Diagnosing Postmortem Failures

9.2.1 Reading Stack Traces

A trace shows the path active at failure, not necessarily the operation that first corrupted state. A crash in a library allocator may have been caused by an earlier overwrite. Follow the invalid state backward through ownership and recent changes rather than declaring the top frame the root cause.

Multiple threads require multiple traces. One thread may be blocked legitimately while another holds a needed resource or has already failed. Correlate traces with logs and timing, and distinguish a deadlock from a slow operation or an intentional wait whose producer never ran.

9.2.2 Analyzing Core Dumps

A core dump can preserve process memory and register state for later analysis, depending on operating-system configuration and limits. It may contain secrets, user data, and credentials, so handle it as a sensitive artifact with controlled access and retention.

Postmortem analysis requires the matching binary and libraries, or enough build identity to recover them. A dump from one release opened against another is not trustworthy evidence. Collection policies, disk limits, and symbol retention should be established before an incident rather than improvised after the relevant files have disappeared.

Capturing and Inspecting a Core Dump from a Controlled Crash

Purpose

Postmortem debugging preserves process state after a crash so the failure can be analyzed without reproducing it under an interactive debugger first.

Code or command

```
ulimit -c unlimited
./crash || true
coredumpctl list ./crash 2>/dev/null || ls -l core* 2>/dev/null
gdb ./crash core
(gdb) bt
(gdb) frame 0
(gdb) info locals
```

What the important parts do

``ulimit`` permits core generation where policy allows it. GDB needs the exact executable plus matching core image to resolve addresses into frames and variables; systemd systems may store cores through ``coredumpctl`` instead of the current directory.

Expected behavior or output

The debugger should show the signal that terminated the process and a backtrace rooted at the failing frame, subject to available symbols and system policy.

Concrete experiment

Rebuild the executable without ``-g`` and compare the backtrace. Then restore matching debug symbols and note which diagnostic details return.

Worked example — Inspect a deterministic calculation with a debugger

Trace the implementation

Build the chapter in Debug and launch GDB with the produced executable. A batch session can use `gdb -q -batch -x source/book05/ch09/optional/inspect.gdb build/native/bin/b05_c09` after substituting the actual build path. The command file breaks at `invoice_total`, records a backtrace and arguments, then continues to the normal checked result. LLDB provides analogous breakpoint, frame, and backtrace commands; command spelling is debugger-specific.

At the breakpoint, `shipping` should be five and the `span` should describe three prices. Step into `subtotal` to inspect how the accumulator changes. A conditional breakpoint or a watchpoint on `total` inside its valid lifetime can test a more specific hypothesis. Do not assume a local's address remains meaningful after its frame exits.

The program is correct by its stated fixture, so the exercise teaches observation before a defect is introduced. The supplied command file is an optional investigation aid; the package's automated test report does not claim that GDB or LLDB sessions were executed unless a corresponding log is present.

Complete worked source

The files below are in `source/book05/ch09/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <array>
#include <span>
#include <stdexcept>
long long subtotal(std::span<const int> values) {
    if (values.size() > 1000) throw std::invalid_argument("count");
    long long total = 0;
    for (int value : values) {
        if (value < 0 || value > 1000) throw std::invalid_argument("value");
        total += value;
    }
    return total;
}
long long invoice_total(std::span<const int> values, int shipping) {
    if (shipping < 0 || shipping > 1000) throw
std::invalid_argument("shipping");
    const auto items = subtotal(values);
    return items + shipping;
}
int main() {
    const std::array prices{12, 20, 8};
    const auto total = invoice_total(prices, 5);
    CHECK(total == 45);
}
```

```
std::cout << "invoice=" << total << "\nPASS\n";  
}
```

Optional source: `optional/inspect.gdb`. This file has a separately stated build or execution procedure and is not part of the default build.

```
set pagination off  
break invoice_total  
run  
bt  
info args  
continue  
quit
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 9
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

In a copy of the chapter, change the shipping calculation to subtract instead of add. Use the failing check to reproduce the defect, then stop at the boundary between subtotal and invoice_total and identify the first incorrect value.

Write an evidence checklist for a production core dump that includes binary identity, symbols, library versions, relevant configuration, privacy controls, and retention.

Solution and review notes

The subtotal remains forty, so the first wrong value occurs when shipping is applied. This narrows the fix to the invoice policy rather than the collection traversal. Restore the correct operation and retain the regression case.

A core dump without matching artifacts can still contain clues, but it is weaker evidence. Preserve the exact build and a minimal reproducible input when possible. Protect memory artifacts because a debugging benefit does not remove their confidentiality risk.

Technical references: [GDB LLDB LIFETIME](#)

Chapter 10 — Detecting Memory and Undefined-Behavior Defects

Memory defects often appear far from their cause. Instrumented builds can turn a silent overwrite or invalid lifetime access into a diagnostic closer to the offending operation. This chapter pairs a correct bounded buffer with an explicitly isolated negative example; the unsafe program is excluded from ordinary builds and must never be counted as a passing functional test merely because it terminates.

10.1 Using Compiler Sanitizers

10.1.1 Detecting Invalid Memory Access

AddressSanitizer instruments supported memory operations and manages runtime metadata to detect classes such as out-of-bounds accesses and use-after-free. Build and link the relevant code with compatible instrumentation so the runtime is available. Its diagnostics identify observed violations, not every possible memory error in unexecuted paths.

Instrumentation changes memory layout, timing, and resource use. A defect that disappears under instrumentation is not thereby disproved. Keep the original reproducer and use multiple evidence sources. Do not use sanitizer benchmark results to estimate ordinary release performance.

10.1.2 Detecting Undefined Behavior

UndefinedBehaviorSanitizer checks selected undefined or suspicious operations, including supported arithmetic and alignment cases. The enabled checks and recovery behavior depend on compiler options and platform. Read the actual diagnostic and the configured check set rather than treating the tool name as complete coverage of the language.

Some defects require other tools. Data races call for a concurrency-aware tool such as ThreadSanitizer on a supported configuration; uninitialized-value tracking has separate tooling and requirements. Incompatible sanitizer combinations should be built in separate directories, not forced together indiscriminately.

Detecting a Heap Buffer Overflow with AddressSanitizer

Purpose

AddressSanitizer is specifically designed to detect invalid memory accesses such as this controlled heap buffer overflow.

Code or command

```
#include <memory>

int main(){
    auto data=std::make_unique<int[]>(4);
    data[4]=99; // intentional one-past-the-end write
}
```

What the important parts do

The allocation owns four `int` elements with valid indices 0 through 3. Writing index 4 crosses the allocation boundary. A sanitizer build instruments the access and maintains metadata describing valid memory regions.

Expected behavior or output

Compile with `-fsanitize=address -fno-omit-frame-pointer`; execution should terminate with a heap-buffer-overflow report that identifies the write site.

Concrete experiment

Change the allocation to five elements and confirm the diagnostic disappears. Then restore four elements and use `data.at` on a `std::vector` to compare library bounds checking with sanitizer instrumentation.

10.2 Detecting Resource Leaks

10.2.1 Identifying Leaked Allocations

A leak occurs when a resource remains allocated after its ownership should have ended. Heap leak detectors can report certain unreachable allocations at process termination, but a still-reachable unbounded cache can exhaust memory without matching that narrow definition. Track both ownership correctness and application-level growth.

Operating-system handles, GPU allocations, and external service resources may need different accounting. A clean heap report does not establish that every file descriptor or CUDA buffer was released. Define a resource inventory and test the relevant failure paths for each type of resource.

10.2.2 Tracking Resource Ownership

RAII makes ordinary cleanup follow object lifetime, reducing the number of paths that need manual release code. A move-only owner expresses transfer; a raw pointer or span normally borrows. Review any escape of a borrowed view beyond the owner's lifetime, even when no explicit delete appears nearby.

Instrument ownership in a focused test when needed. A counted resource wrapper can verify that construction and destruction balance across exceptions. Such a counter is a test of the wrapper's protocol, not a substitute for a memory tool examining the actual allocated storage.

Turning Ownership into RAII to Eliminate a Leak Path

Purpose

Leak detection is most useful when it leads back to an ownership defect. This contrasts manual allocation whose cleanup is skipped during an exception with RAII ownership that cleans up during unwinding.

Code or command

```
#include <memory>
#include <stdexcept>

void unsafe(){
    int* p=new int[1024];
    throw std::runtime_error("failure");
    delete[] p; // unreachable
}

void safe(){
    auto p=std::make_unique<int[]>(1024);
    throw std::runtime_error("failure");
}
```

What the important parts do

``unsafe`` loses its only owning pointer before ``delete[]`` can execute. ``safe`` binds the allocation to a ``unique_ptr``, whose destructor runs automatically when the exception leaves the scope.

Expected behavior or output

A leak checker should report the intentionally unsafe path but not the ``unique_ptr`` allocation in the safe path when each is exercised independently.

Concrete experiment

Write a small harness that calls each function inside ``try/catch``, then run under LeakSanitizer or Valgrind. Replace the raw pointer in ``unsafe`` with ``unique_ptr`` and verify the report changes.

Worked example — Check cleanup and isolate a negative sanitizer fixture

Trace the implementation

The ordinary executable checks deterministic cleanup on normal and exceptional exits. `PlannedFailure` has a distinct type so the catch cannot accidentally swallow a CHECK failure. Tracked counts only this fixture's ownership protocol, while `vector` and `unique_ptr` own the real storage.

Use the root `SERIES_SANITIZERS` option to build the normal examples with AddressSanitizer and UndefinedBehaviorSanitizer on supported GCC or Clang configurations. The optional unsafe source is excluded from that normal build; its purpose is to confirm a diagnostic workflow, not to demonstrate acceptable C++ behavior.

The negative example's index comes from a runtime argument so the experiment does not rely on a compile-time constant access being preserved. Even so, tool behavior must be observed on the actual compiler and platform. The final test report separates normal functional tests from any deliberately failing diagnostic experiments.

Complete worked source

The files below are in `source/book05/ch10/`. The source in this chapter and the companion are exported from the same authored data.

main.cpp

```
#include "check.hpp"
#include <memory>
#include <stdexcept>
#include <vector>
struct Tracked {
    inline static int alive = 0;
    Tracked() { ++alive; }
    ~Tracked() { --alive; }
    Tracked(const Tracked&) = delete;
    Tracked& operator=(const Tracked&) = delete;
};
struct PlannedFailure {};
void operation(bool fail) {
    auto owner = std::make_unique<Tracked>();
    std::vector<int> buffer(8, 3);
    CHECK(buffer.at(7) == 3);
    if (fail) throw PlannedFailure{};
}
int main() {
    operation(false); CHECK(Tracked::alive == 0);
    bool observed = false;
    try { operation(true); } catch (const PlannedFailure&) { observed = true; }
    CHECK(observed && Tracked::alive == 0);
    std::cout << "PASS\n";
}
```

Optional source: optional/unsafe.cpp. This file has a separately stated build or execution procedure and is not part of the default build.

```
// INTENTIONALLY INVALID. Excluded from the normal build and test suite.
// Run only as a separately instrumented negative experiment.
#include <cstdlib>
#include <iostream>
int main(int argc, char** argv) {
    const int index = argc > 1 ? std::atoi(argv[1]) : 8;
    int* values = new int[8]{};
    std::cout << values[index] << '\n'; // index 8 is deliberately out of
    bounds.
}
```

```
    delete[] values;
}
```

Optional source: optional/README.md. This file has a separately stated build or execution procedure and is not part of the default build.

```
# Negative experiment – not a functional example

Use a disposable local build directory and a compiler with AddressSanitizer
support:

```bash
c++ -std=c++20 -g -O1 -fsanitize=address -fno-omit-frame-pointer \
 source/book05/ch10/optional/unsafe.cpp -o build/unsafe-asan
ASAN_OPTIONS=halt_on_error=1 build/unsafe-asan 8
```

Create build first if it does not exist. A supported instrumented run is expected to diagnose the invalid read and exit unsuccessfully. Do not execute the unsafe program without instrumentation or ship it as an application. A missing diagnostic requires investigation; it is not a successful memory-safety result.

```
Run and verify

From the extracted package root:

```bash
python3 tools/run_chapter.py 5 10
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a second owned resource and throw after acquiring both. Verify destruction in reverse lifetime order without catching unrelated test exceptions. Then run the normal fixture under a supported sanitizer configuration and retain the exact command and report.

Explain why `Tracked::alive` equal to zero does not prove that an unrelated raw allocation or file descriptor was released.

Solution and review notes

Each resource needs its own ownership argument. The counter observes only Tracked instances and can be correct while another subsystem leaks. A complete test inventory identifies heap storage, descriptors, locks, threads, and device resources separately.

A sanitizer-clean run establishes that the enabled instrumentation reported no violation on the executed paths. It does not prove absence of undefined behavior

in all inputs, and it does not replace a bounded-input contract or a review of lifetime assumptions.

Technical references: [ASAN](#) [UBSAN](#) [GUIDELINES](#)

Chapter 11 — Improving Code with Static Analysis

Static analysis examines code without requiring every path to execute. Compiler warnings, dedicated analyzers, and guideline checks catch different classes of risk, but each needs the correct compilation context. Harbor’s checked indexing example shows how an explicit interface can eliminate ambiguity before a tool is asked to detect it.

11.1 Treating Compiler Warnings as Engineering Signals

11.1.1 Enabling Strong Warning Levels

Enable a deliberate warning baseline for authored code and review new warnings promptly. Useful diagnostics can expose narrowing conversions, uninitialized values, suspicious conditions, and mismatched declarations. Warning sets differ by compiler and version, so a portable project should express its policy per toolchain rather than copying one universal flag string.

Warnings are not proof of invalid behavior, and silence is not proof of correctness. Investigate the underlying claim. Where a conversion is intentional, make the range argument visible before casting. A cast added only to silence a warning can hide the same defect from both the compiler and the next reviewer.

11.1.2 Removing Warning Debt

A large backlog makes meaningful new warnings easy to miss. Reduce it through focused fixes and a ratchet that prevents new debt. Avoid globally disabling a useful warning because one external header or platform adapter needs a narrow exception.

Treat warnings-as-errors as a build policy with a defined scope. Applying it indiscriminately to every transitive third-party source can make a compiler update break a consumer for reasons outside that consumer’s control. Separate authored targets, vendored code, and generated code while retaining visibility into their risks.

Converting High-Value Compiler Warnings into Build Failures

Purpose

Strong warning levels make suspicious constructs visible during compilation, while `-Werror` can prevent known warning debt from silently entering a maintained target.

Code or command

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion -Werror \
    src/main.cpp -o build/app
```

What the important parts do

`-Wall`, `-Wextra`, and `-Wpedantic` enable broad diagnostic groups; `-Wconversion` adds potentially value-changing implicit conversions; `-Werror` promotes enabled warnings to errors for this build.

Expected behavior or output

The command succeeds only when `src/main.cpp` is free of the enabled diagnostics.

Concrete experiment

Introduce `int x = 3.5;` and observe the conversion diagnostic. Fix the code intentionally instead of suppressing the warning globally.

11.2 Applying Dedicated Static-Analysis Tools

11.2.1 Detecting Correctness Problems

An analyzer needs the actual language mode, macro definitions, include directories, and platform context. A compilation database helps supply per-translation-unit commands. Running a tool against a source file with guessed options can create false reports or miss the code paths used in the real build.

Use findings to generate a concrete hypothesis and regression test. An analyzer warning about a lifetime escape should lead to an ownership review, not just a local suppression. Some analyses are path-sensitive or interprocedural, but their models and resource limits still restrict what they can establish.

11.2.2 Enforcing Coding Guidelines

Style and modernization checks can improve consistency, while correctness checks address different risks. Keep those categories distinguishable so a formatting suggestion does not receive the same urgency as an invalid lifetime. Adopt rules that fit the project's language baseline and supported platforms.

A suppression should be narrow, explained, and revisited when the code or tool changes. Automated rewrites should be reviewed and tested as code changes. A mechanically modern-looking program can still have an incorrect contract, a data race, or an unbounded resource path.

Running clang-tidy with a Project Compilation Database

Purpose

Dedicated static analyzers need the same compile context as the real target so they can reason about includes, definitions, and language mode accurately.

Code or command

```
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
clang-tidy src/parser.cpp -p build \
  --checks='bugprone-*,performance-*,readability-*
```

What the important parts do

`CMAKE\_EXPORT\_COMPILE\_COMMANDS` creates `compile\_commands.json`, and `-p build` tells clang-tidy where to find it. The selected check families target correctness, performance, and maintainability issues beyond normal compiler warnings.

Expected behavior or output

The analyzer emits diagnostics for enabled rules without executing the program.

Concrete experiment

Run once with only `bugprone-\*` and once with the broader set. Categorize which findings are correctness defects versus style or maintainability guidance.

Worked example — Make an indexing contract analyzable

Trace the implementation

The signed index is checked before conversion to `size_t`. That prevents a negative value from becoming a large unsigned index and makes the cast's justification visible. The function returns an optional value rather than an unchecked reference, and `nodiscard` asks callers not to ignore that result accidentally.

Configure the main project with `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON` using a supported generator, then run the optional analysis script with that build directory. The script reports a missing tool as a skip with status 77. It does not manufacture a clean report or substitute a text search for clang-tidy.

The selected checks are a starting configuration, not a certification profile. Review the installed tool's available checks and the project's warning policy before expanding them. The normal runtime fixture verifies the boundary cases independently of whether a static analyzer is available.

Complete worked source

The files below are in `source/book05/ch11/`. The source in this chapter and the companion are exported from the same authored data.

main.cpp

```
#include "check.hpp"
#include <array>
#include <cstdlib>
#include <optional>
#include <span>
[[nodiscard]] std::optional<int> at_checked(std::span<const int> values,
std::ptrdiff_t index) {
    if (index < 0) return std::nullopt;
    const auto converted = static_cast<std::size_t>(index);
    if (converted >= values.size()) return std::nullopt;
    return values[converted];
}
int main() {
    const std::array values{4, 8, 12};
    CHECK(at_checked(values, 0) == 4);
    CHECK(at_checked(values, 2) == 12);
    CHECK(!at_checked(values, -1) && !at_checked(values, 3));
    CHECK(!at_checked({}, 0));
    std::cout << "PASS\n";
}
```

.clang-tidy

```
Checks: '-*,bugprone-*,performance-*,modernize-use-nullptr'
WarningsAsErrors: ''
HeaderFilterRegex: '.*'
FormatStyle: none
```

Optional source: optional/analyze.py. This file has a separately stated build or execution procedure and is not part of the default build.

```
#!/usr/bin/env python3
"""Run an installed clang-tidy against this chapter and a real compilation
database."""
import argparse
from pathlib import Path
import shutil
import subprocess
parser = argparse.ArgumentParser()
parser.add_argument("build_directory", type=Path)
args = parser.parse_args()
if not shutil.which("clang-tidy"):
    print("SKIP: clang-tidy is not installed")
    raise SystemExit(77)
if not (args.build_directory / "compile_commands.json").is_file():
    raise SystemExit("configure a supported generator with
CMAKE_EXPORT_COMPILE_COMMANDS=ON")
```

```
source = Path(__file__).resolve().parents[1] / "main.cpp"
raise SystemExit(subprocess.run(["clang-tidy", str(source), "-p",
str(args.build_directory)]).returncode)
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 11
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a compile-time or review rule that discourages discarding validation results. Demonstrate a warning in a separate copy by ignoring `at_checked`, then restore the correct call and retain the compiler command.

Run analysis on an implementation with a genuinely escaping reference and explain the lifetime defect even if the tool does not report it.

Solution and review notes

A diagnostic is evidence about the configured tool and source, while the lifetime argument follows the language contract. The referent must outlive every use of a borrowed reference; no warning setting can extend it.

For an intentional suppression, record the invariant that makes the operation valid and a test that exercises the boundary. Broad suppression without a reason creates warning debt rather than resolving it.

Technical references: [CLANGTIDY](#) [GCC](#) [CLANG GUIDELINES](#)

Chapter 12 — Measuring Performance with Benchmarks

A benchmark is an experiment whose result depends on its workload and measurement boundary. The question is not whether a number looks small, but whether two implementations were asked to perform equivalent work under comparable conditions. This chapter compares two reductions, preserves a correctness checksum, and reports repeated timings without asserting that either implementation must win.

12.1 Designing Reliable Microbenchmarks

12.1.1 Preventing Compiler Elimination

The compiler is allowed to remove work whose result cannot affect observable behavior. Consume the result in a way the compiler must preserve and provide inputs that represent runtime work. Merely placing a timer around a pure expression does not guarantee the intended instructions remain.

A checksum helps preserve useful computation, but it is not a magical optimization barrier. Repeated identical calls may still be hoisted or combined. Inspect optimized code and vary inputs in a controlled way when needed. Established benchmark libraries provide specialized mechanisms, but those mechanisms also require understanding their scope.

12.1.2 Measuring Repeated Operations

Very short operations can be smaller than timer overhead or resolution. Measure a batch, repeat independent samples, and report the batch definition. Separate setup costs only when the real workload also amortizes them; otherwise excluding allocation or conversion answers a different question from end-to-end performance.

Warmup, cache state, CPU frequency, background load, and thermal behavior can affect results. Keep a record of the environment and avoid pretending that one run represents a universal constant. The example's data mutation is included in both measured paths so it does not secretly favor one implementation.

Preventing Benchmark Elimination with an Observable Result

Purpose

A microbenchmark must make the measured computation observable so an optimizing compiler cannot legally remove it as dead work.

Code or command

```
#include <chrono>
#include <stdint>
#include <iostream>

int main(){
    constexpr std::uint64_t N=50'000'000; volatile std::uint64_t
    sink=0;
    auto start=std::chrono::steady_clock::now();
    std::uint64_t sum=0; for(std::uint64_t i=0;i<N;++i) sum += i;
    sink=sum;
    auto
    ns=std::chrono::duration_cast<std::chrono::nanoseconds>(std::chrono
    ::steady_clock::now()-start).count();
    std::cout<<"result="<<sink<<" ns="<<ns<<"\n";
}
```

What the important parts do

The sum is computed inside the timed region and assigned to a volatile sink afterward, creating an observable side effect. `steady_clock` avoids wall-clock adjustments, but timer overhead and system noise still affect the measurement.

Expected behavior or output

The result field is deterministic for the chosen N; elapsed nanoseconds vary by machine and run.

Concrete experiment

Compile with `-O0` and `-O3`, repeat at least 20 times, and compare distributions rather than selecting the single fastest run.

12.2 Interpreting Benchmark Results

12.2.1 Understanding Variance

Variation is information about the experiment. Report multiple samples and a summary such as a median, while retaining the raw values for investigation. An isolated minimum can approximate an uncontended case but does not describe tail latency experienced by users.

Compare differences with the observed noise. A one-percent change inside a ten-percent spread is weak evidence. Repeated runs, controlled ordering, and representative input distributions make a stronger case than adding decimal places to a single measurement.

12.2.2 Comparing Implementations Fairly

Both implementations must satisfy the same numerical and error contract. Changing precision, skipping validation, or ignoring an inconvenient tail can make a benchmark faster by performing a different task. Verify outputs before interpreting timings.

Choose the relevant build configuration. Debug iterators, assertions, sanitizers, and optimization levels can change the comparison. Keep correctness checks in the benchmark harness, but distinguish harness cost from the operation under study and document exactly what the timed region includes.

Reporting Median and Tail Values Instead of One Benchmark Time

Purpose

Benchmark interpretation requires describing a distribution because individual timings contain scheduler, cache, and system noise.

Code or command

```
#include <algorithm>
#include <iostream>
#include <vector>

int main(){
    std::vector<long long> ns{102,99,101,500,100,98,103,97,105,100};
    std::sort(ns.begin(),ns.end());
    auto median=(ns[4]+ns[5])/2;
    auto p90=ns[8];
    std::cout<<"median="<<median<<" p90="<<p90<<"
    max="<<ns.back()<<"\n";
}
```

What the important parts do

Sorting permits percentile extraction from a fixed sample set. The median describes central tendency more robustly than the 500 ns outlier; the upper percentile and maximum preserve information about slower observations.

Expected behavior or output

The program reports a median near 100 ns while the maximum remains 500 ns, showing why one statistic cannot describe the whole distribution.

Concrete experiment

Add five more 500 ns samples and recompute. Observe how median and p90 respond differently to a systematic tail-latency increase.

Worked example — Compare ordinary and four-lane reductions

Trace the implementation

Both functions sum the same bounded integer data into long long. The four-lane version maintains independent accumulators before combining them and handles the remaining tail explicitly. The input length is intentionally not a multiple of four, so an implementation that ignores the tail fails the comparison.

Each timed batch mutates one input value per repetition and consumes every returned sum in a checksum. The vector copy occurs before the timer starts in

both paths; this experiment therefore measures repeated reduction over an owned working buffer, not end-to-end buffer acquisition. State that boundary when reporting results.

The default harness is primarily a reproducible teaching experiment. Its fixed execution order can bias cache or frequency conditions, and aggressive optimization still warrants assembly inspection. The output reports actual times without a speed assertion. Rebuild with Release for a performance study and retain the Debug or sanitizer build for defect detection.

Complete worked source

The files below are in `source/book05/ch12/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdint>
#include <span>
#include <vector>
long long scalar(std::span<const int> values) {
    long long sum = 0; for (int value : values) sum += value; return sum;
}
long long lanes(std::span<const int> values) {
    long long a=0, b=0, c=0, d=0;
    std::size_t i=0;
    for (; i+4 <= values.size(); i+=4) {
        a+=values[i]; b+=values[i+1]; c+=values[i+2]; d+=values[i+3];
    }
    long long sum=a+b+c+d;
    for (; i<values.size(); ++i) sum+=values[i];
    return sum;
}
struct Measurement { double milliseconds; long long checksum; };
template<class Function>
Measurement measure(std::vector<int> values, Function function) {
    long long checksum=0;
    const auto start=std::chrono::steady_clock::now();
    for (int repetition=0; repetition<64; ++repetition) {
        values[0]=repetition;
        checksum+=function(values);
    }
    const auto elapsed=std::chrono::steady_clock::now()-start;
    return {std::chrono::duration<double,std::milli>(elapsed).count(),checksum};
}
int main() {
    std::vector<int> values(8195);
    for (std::size_t i=0;i<values.size();++i) values[i]=static_cast<int>(i%97);
    CHECK(scalar(values)==lanes(values));
    std::array<double,7> first{}, second{};
    long long checksum=0;
    for (std::size_t round=0;round<first.size();++round) {
        const auto a=measure(values,scalar), b=measure(values,lanes);
        CHECK(a.checksum==b.checksum);
        first[round]=a.milliseconds; second[round]=b.milliseconds;
        checksum=a.checksum;
        std::cout << "round=" << round << " scalar_ms=" << first[round]
```

```

        << " lanes_ms=" << second[round] << '\n';
    }
    std::sort(first.begin(),first.end());
std::sort(second.begin(),second.end());
    std::cout << "median_scalar_ms=" << first[3] << " median_lanes_ms=" <<
second[3]
        << " checksum=" << checksum << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 12
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Alternate the order of the two implementations on successive rounds, retain all samples, and compare the resulting distributions. Add empty, one-element, and several tail lengths to the correctness tests outside the timed region.

Measure a second experiment that includes allocation and copying. Explain why its result answers a different operational question.

Solution and review notes

Alternating order reduces one systematic bias but does not eliminate all environmental effects. Empty input also requires adapting measure, which currently writes values[0] under a nonempty fixture precondition. Test the reduction functions directly for that case rather than violating the harness's own contract.

Including allocation measures the full ownership-acquisition path. An application that reuses buffers may care more about the first experiment; a one-shot request may care about the second. Neither boundary is universally correct independent of the workload.

Technical references: [CHRONO](#) [OPTIMIZE](#) [BENCHMARK](#)

Chapter 13 — Profiling CPU Performance

A benchmark compares selected alternatives; a profiler helps discover where a real workload spends resources. Begin with a representative task and a correctness result, then use samples or counters to investigate a suspected bottleneck. Harbor’s lookup workload deliberately separates linear scanning from indexed lookup so the call graph has a meaningful architectural question behind it.

13.1 Locating Runtime Hotspots

13.1.1 Measuring Function-Level CPU Time

A sampling profiler periodically records where execution is observed. A function receiving many samples is a candidate for investigation, not automatic proof that its source is inefficient. It may be called frequently because of an upstream design choice, or it may be doing necessary work for a dominant workload.

CPU time and wall-clock time differ. A blocked request can have high latency with little CPU consumption, while multiple busy threads can accumulate more CPU time than elapsed time. Choose the measurement that matches the symptom and include waiting, I/O, and queueing when investigating user-visible delay.

13.1.2 Inspecting Call Graphs

A call graph connects expensive work to its callers. Inclusive cost includes descendants; exclusive cost refers to the function itself under the tool’s model. Optimized inlining and tail calls can change the apparent structure, so retain symbols and understand the profiler’s unwinding method.

Use the graph to locate the responsible decision. Repeatedly sorting an unchanged collection may make sort look hot, but the better fix could be moving sorting out of the request loop. Optimize the repeated work pattern before rewriting a well-tested library algorithm.

Collecting Function-Level CPU Samples with perf record

Purpose

Sampling profiles locate where CPU time is actually spent instead of guessing from source complexity.

Code or command

```
g++ -O2 -g workload.cpp -o workload
perf record -g -- ./workload
perf report --stdio --sort comm,dso,symbol | head -40
```

What the important parts do

`perf record -g` samples execution and captures call chains while the workload runs. `perf report` aggregates samples by symbol; debug information and preserved frame information can improve attribution depending on toolchain settings.

Expected behavior or output

The report ranks functions by sampled CPU activity. Percentages describe that measured workload, not intrinsic function cost.

Concrete experiment

Change the input so a different code path dominates and profile again. Confirm the hotspot ranking follows workload behavior rather than remaining fixed.

13.2 Understanding Hardware Performance Counters

13.2.1 Measuring Cache Misses

Hardware counters expose selected microarchitectural events, subject to processor support and operating-system permissions. A cache-miss count can help explain a memory-access pattern, but event names and meanings vary by processor. Multiplexed measurements may be scaled estimates rather than simultaneous exact counts.

Normalize counters against useful work and compare equivalent inputs. A lower miss count can accompany a slower program if it executes more instructions or reduces parallelism. Use counters as evidence within a model of the workload, not as isolated scores to minimize.

13.2.2 Measuring Branch Behavior

Branch-related events can reveal difficult-to-predict control flow, but source-level if statements do not map one-to-one to branch instructions. Compilers can use conditional moves, vector masks, or algebraic transformations. Inspect generated code before attributing a counter change to a particular line.

A branchless rewrite may do extra work on every element. Whether that tradeoff helps depends on input distribution and target hardware. Preserve the same semantics, measure representative distributions, and avoid extrapolating one machine's counter behavior to every supported processor.

Measuring Cache and Branch Events with perf stat

Purpose

Hardware performance counters turn architectural hypotheses about caches and branches into measured event counts for one workload.

Code or command

```
perf stat -e cycles,instructions,cache-references,cache-  
misses,branches,branch-misses \  
./workload
```

What the important parts do

`cycles` and `instructions` support IPC-style ratios; cache and branch events reveal miss behavior. Event definitions and availability are processor-specific, so counters must be interpreted with the exact hardware and kernel context.

Expected behavior or output

`perf stat` prints event counts and derived percentages where supported.

Concrete experiment

Run the same binary with sequential and randomized access patterns. Compare cache-miss rates while keeping total work comparable.

Worked example — Profile equivalent lookup strategies

Trace the implementation

The fixture creates a sorted collection and a deterministic mixture of present and absent keys. Both lookup functions count the same hits. The `-profile` option increases work for a profiler without changing the data contract; ordinary CTest execution uses the short mode.

On a Linux host with perf installed and permission to collect the requested events, a starting experiment is `perf stat -- build/native/bin/b05_c13 --profile`. A call-graph experiment can use `perf record` with a supported unwinding configuration. Substitute the actual binary path and consult the installed tool for event and permission details. The book does not require changing global kernel security settings to run a profile.

The binary-search path relies on sorted input, a precondition established by construction. Its lower comparison growth is an algorithmic advantage, but actual timing still depends on access patterns and query distribution. The checksum prevents the entire workload from becoming irrelevant to program output, while profiling an optimized build shows what the compiler actually emitted.

Complete worked source

The files below are in `source/book05/ch13/`. The source in this chapter and the companion are exported from the same authored data.

main.cpp

```
#include "check.hpp"
#include <algorithm>
#include <cstdint>
#include <cstdint>
#include <numeric>
#include <span>
#include <string_view>
#include <vector>
std::size_t scan_hits(std::span<const int> values, std::span<const int> queries)
{
    std::size_t hits=0;
    for (int query:queries) if (std::find(values.begin(),values.end(),query)!=
values.end()) ++hits;
    return hits;
}
std::size_t binary_hits(std::span<const int> values, std::span<const int>
queries) {
    std::size_t hits=0;
    for (int query:queries) if
(std::binary_search(values.begin(),values.end(),query)) ++hits;
    return hits;
}
int main(int argc,char** argv) {
    const bool profile=argc==2 && std::string_view(argv[1])=="--profile";
    if (argc>1 && !profile) return 2;
    std::vector<int> values(4096); std::iota(values.begin(),values.end(),0);
    std::vector<int> queries(1024);
    std::uint32_t state=17;
    for (int& query:queries) { state=state*1664525u+1013904223u;
query=static_cast<int>(state%8192u); }
    const auto expected=scan_hits(values,queries);
    CHECK(binary_hits(values,queries)==expected);
    std::size_t checksum=0;
    const int rounds=profile?200:2;
    for (int round=0;round<rounds;++round) {
        queries[0]=round%8192;
        const auto a=scan_hits(values,queries), b=binary_hits(values,queries);
        CHECK(a==b); checksum+=a+b;
    }
}
```



```
std::cout << "rounds=" << rounds << " checksum=" << checksum << "\nPASS\n";  
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 13
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Separate the two lookup modes into individually selectable workloads so their counters can be compared without mixing costs. Preserve a correctness oracle outside the timed region.

Try mostly-present and mostly-absent query sets, then record how input distribution changes the scan's work and the branch behavior. Do not present one distribution as representative without justification.

Solution and review notes

A separate mode makes attribution clearer, but it must process the same query bytes and report the same hit count. Linear search can terminate early on present keys, while absent keys scan the entire sequence. That difference explains why a distribution change affects its work even before considering hardware counters.

Retain raw profiler output with compiler flags and machine details. A missing or permission-denied counter is unavailable evidence, not a zero count. Report that limitation explicitly rather than filling in an expected value.

Technical references: [PERF ALGORITHMS OPTIMIZE](#)

Chapter 14 — Understanding Compiler Optimization

Optimization preserves the program’s required observable behavior while transforming its implementation. That promise is only meaningful for behavior the language actually defines. A compiler can exploit undefined behavior in ways that make a seemingly harmless source change produce surprising machine code, so correctness and optimization analysis cannot be separated.

14.1 Understanding Optimization Levels

14.1.1 Comparing Debug and Optimized Builds

Optimization levels select groups of transformations whose exact contents vary by compiler and release. Debug builds favor convenient inspection; optimized builds favor generated-code quality under the chosen settings. Neither name alone tells you every active option, so retain the actual command line.

Some flags alter semantic assumptions rather than merely rearranging equivalent work. Fast-math options can relax floating-point rules, and architecture-specific options can emit instructions unavailable on deployment machines. Treat those choices as compatibility and numerical-contract decisions, not free performance switches.

14.1.2 Understanding Inlining and Constant Folding

Inlining replaces a call boundary with an opportunity to optimize the callee in context; it does not mean every function marked inline must be expanded. The inline keyword also has language linkage and definition implications that are distinct from optimizer heuristics.

Constant folding evaluates known expressions during translation when permitted. Propagation can expose further simplification across loops and branches. A benchmark using only compile-time-known inputs may therefore measure little or none of the computation the author expected. Inspect the generated artifact instead of inferring instructions from the source’s visual length.

Comparing Debug and Optimized Compiler Output

Purpose

Optimization levels change compiler transformation policy, not the program's intended semantics. Building both configurations lets the reader compare artifact size, debug characteristics, and measured runtime.

Code or command

```
g++ -std=c++20 -O0 -g compute.cpp -o compute_00
g++ -std=c++20 -O3 -DNDEBUG compute.cpp -o compute_03
size compute_00 compute_03
/usr/bin/time -f '%e s' ./compute_00
/usr/bin/time -f '%e s' ./compute_03
```

What the important parts do

`-O0 -g` favors source-level debugging; `-O3` enables aggressive optimization and `-DNDEBUG` disables standard assertions. `size` compares sections, while `/usr/bin/time` gives a coarse runtime observation.

Expected behavior or output

Both programs should produce the same functional output; binary size and runtime can differ.

Concrete experiment

Add a deterministic output checksum and verify equality across builds before treating any runtime difference as a performance improvement.

14.2 Inspecting Generated Machine Code

14.2.1 Reading Compiler Assembly Output

Assembly inspection helps connect source assumptions to actual loads, arithmetic, branches, and calls. Start with a small function and a specific question, such as whether a loop vectorized or an abstraction introduced a call. You do not need to reverse-engineer an entire application to test one hypothesis.

Use the exact target architecture and flags under investigation. Source-to-assembly output and disassembly of the linked binary answer related but different questions because linking and link-time optimization may transform code further. Keep symbols and identify the artifact you inspected.

14.2.2 Verifying Optimization Assumptions

A comment saying branchless or vectorized is not evidence that the emitted code has that property. Compiler optimization reports and disassembly can support the claim, while a benchmark determines whether the change helps the relevant workload. Both are needed before attributing a speedup to a particular mechanism.

Avoid forcing an instruction pattern merely because it looks efficient in isolation. Register pressure, code size, dependency chains, and surrounding work can change the result. Prefer a clear baseline, measure a candidate, and retain the simpler implementation when the evidence does not justify added complexity.

Inspecting Constant Folding and Inlining in Generated Assembly

Purpose

Generated assembly is direct evidence for whether a specific optimization hypothesis, such as inlining or constant folding, occurred in a particular build.

Code or command

```
g++ -std=c++20 -O0 -S -masm=intel optimize.cpp -o 00.s
g++ -std=c++20 -O3 -S -masm=intel optimize.cpp -o 03.s
diff -u 00.s 03.s | less
```

What the important parts do

`-S` stops after assembly generation and `-masm=intel` selects one readable syntax. Building twice with different optimization levels isolates the compiler policy change; `diff` reveals which calls or calculations disappeared or changed.

Expected behavior or output

The optimized assembly is often shorter around constant expressions and small functions, though exact instructions are compiler- and architecture-specific.

Concrete experiment

Mark a small function `noinline`, rebuild at `-O3`, and verify that the call reappears. Then remove the attribute and compare again.

Worked example — Compare two defined formulations before inspecting assembly

Trace the implementation

Both formulations operate in unsigned 64-bit arithmetic, whose modular behavior is defined. For the small supplied inputs the result is also the ordinary mathematical sum. At enormous input sizes, modular wrap is part of these function signatures rather than an accidental signed-overflow assumption; a production exact-total API would need an overflow policy.

The optional shell script emits unoptimized and optimized assembly into a caller-selected output directory. It uses the same source and include path for both. It does not enable a machine-specific ISA or claim that one particular transformation must occur. CXX can select a compatible GCC or Clang driver.

The constant fixture in main can itself be simplified, so inspect the named weighted and factored functions as well as their callers. Link-time optimization can further alter the final executable, which is why a compiler -S output is not always the last word on deployed instructions.

Complete worked source

The files below are in `source/book05/ch14/`. The source in this chapter and the companion are exported from the same authored data.

main.cpp

```
#include "check.hpp"
#include <array>
#include <cstdlib>
#include <span>
std::uint64_t weighted(std::span<const std::uint32_t> values) {
    std::uint64_t sum=0;
    for (auto value:values) sum+=static_cast<std::uint64_t>(value)*3u+1u;
    return sum;
}
std::uint64_t factored(std::span<const std::uint32_t> values) {
    std::uint64_t sum=0;
    for (auto value:values) sum+=value;
    return sum*3u+values.size();
}
int main() {
    const std::array<std::uint32_t,7> values{0,1,2,3,10,100,1000};
    CHECK(weighted(values)==factored(values));
    CHECK(weighted({})==0 && factored({})==0);
    std::cout << "result=" << weighted(values) << "\nPASS\n";
}
```

Optional source: optional/inspect.sh. This file has a separately stated build or execution procedure and is not part of the default build.

```
#!/usr/bin/env bash
set -euo pipefail
root=$(cd -- "${dirname -- "${BASH_SOURCE[0]}"}/../.." && pwd)
out=${1:?Usage: inspect.sh OUTPUT_DIRECTORY}
```

```
mkdir -p -- "$out"
compiler=${CXX:-c++}
"$compiler" -std=c++20 -O0 -S -I"$root/support" "$root/book05/ch14/main.cpp" -o
"$out/unoptimized.s"
"$compiler" -std=c++20 -O3 -S -I"$root/support" "$root/book05/ch14/main.cpp" -o
"$out/optimized.s"
printf '%s\n' "Assembly written to $out; compare the named functions, not just
file sizes."
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 14
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Inspect whether the compiler transforms weighted into a form resembling factored, then benchmark both with runtime data. Record a case where identical source-level work produces different code under different flags.

Change the contract to checked exact arithmetic and identify which algebraic transformations remain valid when intermediate overflow must be reported rather than wrapped.

Solution and review notes

Unsigned modular addition and multiplication support the distributive transformation used here. Checked arithmetic adds observable error behavior, so changing the order of intermediate checks can change which inputs are accepted or which failure is reported.

An assembly difference without a timing difference may be irrelevant to the workload, and a timing difference without an understood mechanism should be investigated rather than assigned a convenient explanation. Preserve both artifacts and measurements.

Technical references: [OPTIMIZE](#) [GCC](#) [CLANG](#) [NUMERIC](#)

Chapter 15 — Understanding CPU Architecture for C++ Performance

Modern processors execute source code through several layers of translation, scheduling, caching, and speculation. A useful performance model identifies dependencies and data movement without pretending that every processor shares the same pipeline or cache geometry. This chapter compares one long dependency chain with several independent accumulators while keeping the numerical result exact for the chosen input.

15.1 Understanding Modern Processor Pipelines

15.1.1 Understanding Instruction-Level Parallelism

Independent instructions can overlap in execution when the processor and generated code permit it. A reduction that repeatedly updates one accumulator creates a dependency chain; multiple partial accumulators can expose independent work before a final combination.

The benefit is limited by available execution resources, register pressure, compiler transformations, and the workload's actual bottleneck. A memory-bound loop may not improve merely because it has more arithmetic independence. Inspect and measure rather than assuming that additional accumulators always help.

15.1.2 Understanding Out-of-Order Execution

Many high-performance CPUs can schedule ready operations before earlier stalled operations while preserving required architectural behavior. This can hide some latency when independent work is available. It cannot remove true data dependencies or make an invalid data race legal.

C++ ordering rules, compiler reordering, and processor execution are distinct layers. Synchronization must be expressed using the language's atomics or locks, not by assuming source order becomes a fixed instruction schedule. A performance explanation should not turn an observed processor behavior into a portability guarantee.

Exposing Instruction-Level Parallelism with Independent Accumulators

Purpose

Independent dependency chains give an out-of-order processor more operations that can be in flight simultaneously than one long serial accumulator chain.

Code or command

```
#include <cstdint>
#include <iostream>
#include <vector>

int main(){
    std::vector<int> v(1'000'000,1); long long a=0,b=0,c=0,d=0;
    for(std::size_t i=0;i+3<v.size();i+=4)
        {a+=v[i];b+=v[i+1];c+=v[i+2];d+=v[i+3];}
    std::cout<<a+b+c+d<<"\n";
}
```

What the important parts do

The loop updates four separate sums, so each iteration exposes independent additions. The compiler may further unroll or vectorize, which is why assembly inspection and counters are needed to confirm the mechanism on a real build.

Expected behavior or output

The program prints `1000000`.

Concrete experiment

Compare against one accumulator with vectorization disabled. Measure cycles per element and inspect assembly before attributing any difference specifically to instruction-level parallelism.

15.2 Understanding Memory Hierarchies

15.2.1 Understanding CPU Cache Levels

Caches retain recently used memory blocks closer to execution resources. Their sizes, associativity, sharing, and line geometry vary by processor. Contiguous access often uses fetched data efficiently, while pointer chasing can make each next address depend on a prior load.

The working set matters: a layout that fits one cache on a test machine may not fit on another. Record input size and access pattern with the hardware description. Avoid teaching one cache-line size or miss latency as a universal C++ property.

15.2.2 Understanding Main-Memory Latency

A cache miss can require access to a more distant cache or main memory, with latency affected by contention, topology, and access pattern. Multiple outstanding requests can improve throughput, but a dependent pointer chain may expose latency directly because the next address is not known yet.

Bandwidth and latency are different constraints. Streaming a large array may approach a transfer-rate limit, while repeatedly following dependent links may process few bytes yet remain slow. Choose data structures and work partitioning based on which constraint the measurement supports.

Contrasting Sequential and Strided Memory Access

Purpose

Cache hierarchies reward spatial and temporal locality. A large-stride traversal deliberately uses only one element from many cache lines, increasing the amount of transferred memory per useful integer.

Code or command

```
#include <iostream>
#include <vector>

int main(){
    constexpr std::size_t N=1<<22; std::vector<int> v(N,1); long long
    sum=0;
    for(std::size_t i=0;i<N;i+=16) sum+=v[i];
    std::cout<<sum<<"\n";
}
```

What the important parts do

The vector is contiguous, but stepping by 16 integers typically skips most values in each fetched cache line. The sum remains correct for the selected elements; the performance cost depends on cache line size, prefetching, and memory hierarchy.

Expected behavior or output

For $N=2^{22}$ and stride 16, the result equals the number of visited elements.

Concrete experiment

Benchmark strides 1, 2, 4, 8, 16, 32, and 64 while reporting nanoseconds per visited element. Correlate changes with cache-miss counters.

Worked example — Expose independent reduction chains

Trace the implementation

The test covers all lengths from zero through 129, including every possible tail modulo eight. Both algorithms use unsigned 64-bit accumulation and receive small values, so the expected equality is not clouded by floating-point reassociation. This separates a structural dependency experiment from a numerical-accuracy experiment.

The independent version exposes eight chains, but the compiler may unroll, vectorize, or otherwise transform either function. The source alone does not prove the final number of hardware chains. Apply Chapter 14's inspection workflow and Chapter 12's benchmark discipline before making a performance claim.

The fixture does not time the tiny cases because their main purpose is correctness coverage. A performance study should use runtime-created working sets spanning relevant cache sizes, preserve output checks, and report the machine. A different processor can legitimately prefer a different formulation.

Complete worked source

The files below are in `source/book05/ch15/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <array>
#include <cstdint>
#include <cstdlib>
#include <span>
#include <vector>
std::uint64_t serial_sum(std::span<const std::uint32_t> values) {
    std::uint64_t sum=0; for (auto value:values) sum+=value; return sum;
}
std::uint64_t independent_sum(std::span<const std::uint32_t> values) {
    std::array<std::uint64_t,8> lanes{};
    std::size_t i=0;
    for (;i+8<=values.size();i+=8)
        for (std::size_t lane=0;lane<8;++lane) lanes[lane]+=values[i+lane];
    std::uint64_t sum=0; for (auto lane:lanes) sum+=lane;
    for (;i<values.size();++i) sum+=values[i];
    return sum;
}
int main() {
    for (std::size_t size=0;size<130;++size) {
        std::vector<std::uint32_t> values(size);
        for (std::size_t i=0;i<size;++i)
            values[i]=static_cast<std::uint32_t>(i*7);
        CHECK(serial_sum(values)==independent_sum(values));
    }
    std::cout << "tested_lengths=130\nPASS\n";
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 15
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Benchmark several accumulator counts on the same runtime input and inspect whether the compiler uses scalar or vector instructions. Repeat with working sets below and above a relevant cache capacity measured on your machine.

Explain why a dependent linked traversal may not benefit from the same accumulator technique.

Solution and review notes

Independent accumulators break arithmetic dependencies, not address dependencies. In a linked traversal, the next pointer may require the current load to complete, leaving less independent work to overlap. A contiguous representation or batched traversal can change that structure more fundamentally.

The best accumulator count is an empirical result for a toolchain and workload. Too many live values can increase register pressure or code size. Keep the simplest version that meets the measured requirement.

Technical references: [CPUARCH](#) [OPTIMIZE](#) [NUMERIC](#)

Chapter 16 — Designing Cache-Friendly Software

Data layout should follow the operations that dominate a workload. Harbor’s sample records contain a frequently processed measurement and less frequently used metadata. Comparing arrays of structures with structures of arrays reveals a concrete question: how many irrelevant bytes must the processor bring through its memory hierarchy to perform the selected transformation?

16.1 Improving Data Locality

16.1.1 Improving Spatial Locality

Spatial locality means using nearby data while the fetched region is useful. Contiguous containers often help sequential traversal because adjacent elements occupy adjacent storage. A pointer-rich structure can scatter logically related values across allocations, making a traversal require more independent memory fetches.

Locality is workload-specific. An array of complete records can be excellent when every operation needs all fields, while a columnar layout can be better when an operation touches only one field across many records. Count the fields actually used before adopting a layout based on a slogan.

16.1.2 Improving Temporal Locality

Temporal locality means reusing data before it is displaced from fast storage. Blocking or tiling can keep a working subset active while performing several operations on it. Recomputing or reloading a large intermediate between distant stages can lose that opportunity.

Fusion can improve reuse, but it can also increase live state, register pressure, or coupling between stages. Compare the complete pipeline, including any conversion cost needed to enter the new layout. An optimization that accelerates one loop may slow the surrounding workload.

Improving Spatial Locality by Traversing Contiguous Rows

Purpose

Row-major traversal consumes adjacent elements while they are already in nearby cache lines, demonstrating spatial locality with a simple contiguous representation.

Code or command

```
#include <iostream>
#include <vector>

int main(){
    const int rows=1024, cols=1024; std::vector<int> m(rows*cols,1);
    long long sum=0;
    for(int r=0;r<rows;++r)
        for(int c=0;c<cols;++c)
            sum += m[r*cols+c];
    std::cout<<sum<<"\n";
}
```

What the important parts do

`r*cols+c` maps a two-dimensional logical coordinate into one contiguous row-major vector. Incrementing `c` touches the next integer in memory, allowing each fetched cache line to contribute several useful values.

Expected behavior or output

The program prints `1048576`.

Concrete experiment

Swap the loop order so columns are outermost. Benchmark both orders for larger matrices and compare cache-miss counts.

16.2 Choosing Better Data Layouts

16.2.1 Comparing Arrays of Structures

An array of structures keeps each entity's fields together. It is natural for object-level access, serialization adapters, and code that repeatedly needs complete records. Its stride includes all fields and padding, even when a particular loop uses only one measurement field.

Do not infer the stride by adding visible member sizes; inspect `sizeof` and the actual layout constraints. Avoid serializing the raw object representation merely because records are contiguous. Padding, byte order, and type representation remain separate portability concerns.

16.2.2 Comparing Structures of Arrays

A structure of arrays stores each field in its own contiguous sequence. A loop over one field can then traverse a dense stream of relevant values, and vectorization may become easier. The tradeoff is maintaining correspondence between columns and handling operations that need several fields together.

Encapsulate the equal-length invariant rather than exposing unrelated public vectors that can drift apart. Define insertion, removal, and stable-identity behavior explicitly. Index-based correspondence is a representation decision, not an excuse to abandon domain invariants.

Comparing Array-of-Structures with Structure-of-Arrays Layout

Purpose

When a workload processes only one field, structure-of-arrays can pack that field contiguously instead of dragging unrelated members through the memory hierarchy.

Code or command

```
#include <iostream>
#include <vector>

struct Particle{float x,y,z,mass;};
int main(){
    std::vector<Particle> aos(1000); for(auto& p:aos)p.x=1.0f;
    std::vector<float> x(1000,1.0f), y(1000), z(1000), mass(1000);
    float sa=0,ss=0; for(auto& p:aos)sa+=p.x; for(float v:x)ss+=v;
    std::cout<<sa<<' '<<ss<<"\n";
}
```

What the important parts do

The AoS vector interleaves x, y, z, and mass for each particle. The SoA representation gives x its own dense array. Both represent equivalent logical data but create different memory-access patterns.

Expected behavior or output

Both sums print `1000`.

Concrete experiment

Expand each particle with additional cold fields and benchmark an x-only transform. Then benchmark a workload that needs all fields to see when AoS can become competitive again.

Worked example — Convert records to validated columns

Trace the implementation

Columns owns its two sequences privately and builds them from one accepted record span. Construction failure destroys the partially constructed object before it is published. The exposed view is read-only and remains valid only while the Columns object and its storage remain alive.

The energy calculation uses the same double-precision accumulation order in both layouts. The selected integer-valued floats produce an exactly representable result, making the fixture's equality check appropriate. A broader numerical workload would need finite-input validation and a tolerance policy.

The printed record size is observed on the current implementation, not hard-coded as a universal stride. The example proves equivalence of the selected transformation and exposes layout size; it does not prove a speedup. A benchmark must include representative data volume and decide whether conversion into columns is amortized across repeated operations.

Complete worked source

The files below are in `source/book05/ch16/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <array>
#include <cstdint>
#include <span>
#include <stdexcept>
#include <vector>
struct Sample { std::uint32_t id; float value; std::array<char,48> note; };
class Columns {
    std::vector<std::uint32_t> ids_;
    std::vector<float> values_;
public:
    explicit Columns(std::span<const Sample> samples) {
        ids_.reserve(samples.size()); values_.reserve(samples.size());
        for (const auto& sample:samples) { ids_.push_back(sample.id);
values_.push_back(sample.value); }
    }
    std::span<const float> values() const { return values_; }
    std::size_t size() const { return ids_.size(); }
};
double energy_records(std::span<const Sample> samples) {
    double result=0; for (const auto& sample:samples)
result+=static_cast<double>(sample.value)*sample.value;
    return result;
}
double energy_columns(std::span<const float> values) {
    double result=0; for (float value:values)
result+=static_cast<double>(value)*value;
    return result;
}
int main() {
    const std::array samples{Sample{1,2.0f,{}},Sample{2,-3.0f,{}},Sample{3,4.0f,
```

```

    {}));
    const Columns columns{samples};
    CHECK(columns.size()==samples.size());
    CHECK(energy_records(samples)==29.0 &&
energy_columns(columns.values())==29.0);
    CHECK(energy_columns({})==0.0);
    std::cout << "record_bytes=" << sizeof(Sample) << " value_bytes=" <<
sizeof(float) << "\nPASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 16
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add a second frequently processed field and a workload that needs both fields. Compare the cost of two column streams with complete-record access. Include conversion time in a separate end-to-end measurement.

Design an append operation that preserves equal column lengths if allocation fails. Explain why two independent `push_back` calls are not automatically a strong exception guarantee.

Solution and review notes

One approach reserves sufficient capacity for both columns before mutating either, then performs only nonthrowing insertions for the chosen element types. Another builds a candidate representation and commits it atomically at the object level. The correct approach depends on element construction guarantees.

A layout is justified by workload evidence, not by being labeled data-oriented. Keep domain identity and representation invariants intact while changing physical organization.

Technical references: [VECTOR SPAN](#) [CPUARCH](#) [GUIDELINES](#)

Chapter 17 — Accelerating Computation with SIMD

SIMD applies one instruction to multiple data lanes. It can accelerate suitable arithmetic when data layout, dependencies, alignment, and numerical semantics permit it. The worked example provides a portable scalar path and an x86 SSE path selected at compilation, with complete tail handling and no assumption that every machine supports a newer optional instruction set.

17.1 Understanding Vector Execution

17.1.1 Processing Multiple Values per Instruction

A vector instruction operates on several lanes according to an instruction's semantics. This is distinct from creating several operating-system threads. A compiler may vectorize a loop automatically, while explicit intrinsics expose selected instructions or operations through a compiler-specific interface.

Vector width alone does not determine throughput. Loads, stores, arithmetic dependencies, alignment, and target execution resources all matter. A wider instruction can also have different frequency or scheduling consequences on some processors. Measure the complete loop rather than multiplying a scalar timing by the lane count.

17.1.2 Understanding SIMD Register Widths

Instruction-set families define different vector widths and supported element operations. A binary compiled for an optional ISA must not execute unsupported instructions on an older deployment processor. Compile-time selection is sufficient only when the deployment baseline guarantees that ISA.

Runtime dispatch can choose a compatible implementation, but the dispatcher itself must be compiled for the baseline and the feature detection must follow platform rules. Do not compile an entire application with a high-end machine flag and assume one runtime branch makes the rest portable.

Processing Four Floats per SIMD Instruction with SSE

Purpose

SIMD works by applying one instruction to multiple packed lanes. This explicit intrinsic example makes the vector width and lane operation visible.

Code or command

```
#include <immintrin.h>
#include <iostream>

int main(){
    alignas(16) float a[4]{1,2,3,4}, b[4]{10,20,30,40}, out[4]{};
    __m128 va=_mm_load_ps(a); __m128 vb=_mm_load_ps(b);
    __m128 vc=_mm_add_ps(va,vb); _mm_store_ps(out,vc);
    std::cout<<out[0]<<' '<<out[3]<<"\n";
}
```

What the important parts do

`\_\_m128` represents 128 bits, enough for four 32-bit floats. The load intrinsics populate vector registers, `\_mm\_add\_ps` performs four additions, and the store writes all four lane results.

Expected behavior or output

The program prints `11 44` on x86 targets with SSE support.

Concrete experiment

Compile to assembly and locate the vector add instruction. Then implement the same operation as a scalar loop and see whether the optimizer auto-vectorizes it to similar instructions.

17.2 Enabling Vectorization

17.2.1 Writing Auto-Vectorizable Loops

Simple contiguous loops with clear bounds and independent iterations are good candidates for automatic vectorization. Hidden aliasing, function calls, irregular control flow, or loop-carried dependencies can obstruct it. A compiler report can explain a particular decision, but the report must match the actual optimization flags and target.

Do not assert nonaliasing unless the API guarantees it. A false promise can create incorrect optimized code. Similarly, floating-point reassociation may require a changed numerical contract. Preserve semantics first, then identify which constraints can legitimately be relaxed.

17.2.2 Using Explicit SIMD Intrinsics When Necessary

Intrinsics can help when a measured hotspot needs a specific operation the compiler does not generate well. They also increase platform coupling and maintenance cost. Retain a scalar reference, isolate ISA-specific code, and test all remainder lengths and alignment cases.

An unaligned load instruction avoids one alignment precondition but still requires the entire accessed range to be valid. Loading four elements when only one remains is out of bounds even if unused lanes are later ignored. Handle the tail without reading beyond the object.

Writing an Auto-Vectorizable Reduction-Free Transform

Purpose

Simple independent loop iterations give the compiler a strong opportunity to auto-vectorize because there are no loop-carried dependencies.

Code or command

```
#include <cstdint>

void scale(float* __restrict out,const float* __restrict
in,std::size_t n,float factor){
    for(std::size_t i=0;i<n;++i) out[i]=in[i]*factor;
}
```

What the important parts do

Each output element depends only on the matching input element and a scalar factor. The `restrict` extension on compilers that support it communicates non-aliasing, removing one potential barrier to vectorization.

Expected behavior or output

With optimization and a vectorization-capable target, compilers commonly emit packed multiply/store operations; exact instructions depend on architecture and flags.

Concrete experiment

Compile with GCC `-O3 -fopt-info-vec` and inspect the vectorization report. Remove the non-aliasing information or introduce a loop-carried dependency and compare the report.

Worked example — Add vectors with a guarded SIMD implementation

Trace the implementation

The guarded path uses four-lane floating-point loads, addition, and stores on a compilation target that exposes the selected x86 capability. Other targets compile the scalar loop. The output states which path was compiled, so a passing scalar fallback is not misrepresented as an executed SIMD test.

The loop condition ensures four complete elements remain before a vector load. The scalar tail covers every remaining element. The fixture tests all lengths from zero through sixty-four with separate input and output buffers. The API assumes no harmful partial overlap; exact in-place or overlapping policies should be specified and tested before callers rely on them.

These inputs avoid numerical ambiguity. This operation performs the same per-element addition rather than a reassociated reduction, but floating-point environment and exceptional values can still matter for a broader contract. No throughput claim is made by the equality tests alone.

Complete worked source

The files below are in `source/book05/ch17/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <cstdint>
#include <span>
#include <stdexcept>
#include <vector>
#if defined(__SSE2__) || defined(_M_X64)
#include <emmintrin.h>
#define HARBOR_HAS_SSE 1
#else
#define HARBOR_HAS_SSE 0
#endif
void add_scalar(std::span<const float> a, std::span<const float>
b, std::span<float> output) {
    if (a.size()!=b.size() || a.size()!=output.size()) throw
std::invalid_argument("length mismatch");
    for (std::size_t i=0; i<a.size();++i) output[i]=a[i]+b[i];
}
void add_selected(std::span<const float> a, std::span<const float>
b, std::span<float> output) {
    if (a.size()!=b.size() || a.size()!=output.size()) throw
std::invalid_argument("length mismatch");
    std::size_t i=0;
#if HARBOR_HAS_SSE
    for (; i+4<=a.size(); i+=4) {
        const auto x=_mm_loadu_ps(a.data()+i), y=_mm_loadu_ps(b.data()+i);
        _mm_storeu_ps(output.data()+i, _mm_add_ps(x,y));
    }
#endif
    for (; i<a.size();++i) output[i]=a[i]+b[i];
}
```

```
int main() {
    for (std::size_t size=0;size<65;++size) {
        std::vector<float> a(size),b(size),expected(size),actual(size);
        for (std::size_t i=0;i<size;++i) { a[i]=static_cast<float>(i);
        b[i]=static_cast<float>(2*i); }
        add_scalar(a,b,expected); add_selected(a,b,actual);
        CHECK(actual==expected);
    }
    std::cout << "compiled_sse_path=" << HARBOR_HAS_SSE << "\nPASS\n";
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 17
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Test spans beginning one element into a larger allocation to exercise unaligned addresses without going out of bounds. Add mismatched-length rejection and every possible tail length.

Build a baseline scalar target with vectorization disabled for an experiment, then compare it with automatic and explicit vectorization under otherwise equivalent flags. Record the actual deployed ISA requirement.

Solution and review notes

An offset span is valid only when its extent remains inside the owning allocation. Unaligned instructions permit the address alignment, not an invalid extent. The mismatch checks must occur before either implementation writes output.

A fair comparison distinguishes compiler-generated SIMD from the handwritten path. Inspect assembly and preserve the same numerical contract. Keep architecture-specific flags scoped to the intended implementation rather than leaking them into the whole executable.

Technical references: [INTRINSICS](#) [VECTORIZER](#) [CPUARCH](#) [SPAN](#)

Chapter 18 — Applying Data-Oriented Design

Data-oriented design begins with transformations and access patterns. Harbor processes active measurements repeatedly, so it can build a dense active-value sequence once and operate on that sequence instead of repeatedly chasing objects and checking the same state. The chapter compares representations without claiming that object-oriented and data-oriented design are mutually exclusive ideologies.

18.1 Designing Around Data Transformations

18.1.1 Grouping Frequently Processed Data

Identify which fields are read together, which are written together, and how often each transformation runs. Grouping data by that workload can reduce irrelevant memory traffic and simplify loops. A dense active set may be useful when many passes operate on a relatively stable subset.

Maintaining the dense set has a cost. If activity changes constantly or only one pass is needed, rebuilding the representation can outweigh the saved traversal. Include update and conversion work in the lifecycle measurement rather than timing only the favorable inner loop.

18.1.2 Eliminating Pointer-Chasing Structures

Indirection can express flexible relationships, but each pointer load may delay discovery of the next address. Contiguous indices or compact arrays can make traversal more predictable and easier to batch. Replacing pointers also changes lifetime and identity management, which must be designed explicitly.

An index is not automatically a stable identifier. Erasing or reordering a vector can change what an index denotes. Use stable keys, generation-tagged handles, or an indirection table when external references must survive representation changes. Performance work must preserve identity semantics.

Replacing Pointer Chasing with a Dense Transformation Pipeline

Purpose

Data-oriented design starts from the transformation applied to a collection. A dense vector supports filtering and transformation without navigating separately allocated nodes.

Code or command

```
#include <algorithm>
#include <iostream>
#include <vector>

int main(){
    std::vector<float> temperatures{20,21,100,22,23};
    std::erase_if(temperatures,[](float t){return t>60;});

    std::transform(temperatures.begin(),temperatures.end(),temperatures
        .begin(),[](float t){return t*1.8f+32;});
    for(float t:temperatures) std::cout<<t<<' ';
}
```

What the important parts do

``erase_if`` compacts the vector after removing invalid values. ``transform`` then walks the remaining contiguous elements and overwrites each with the converted value. The data flow is explicit and cache-friendly.

Expected behavior or output

The 100-degree outlier is removed; the remaining Celsius values are printed in Fahrenheit.

Concrete experiment

Implement the same workload with a linked list of heap-allocated nodes and profile cache misses and allocation count for a much larger dataset.

18.2 Comparing Object-Oriented and Data-Oriented Models

18.2.1 Evaluating Abstraction Costs

An abstraction's cost depends on its implementation and use. A small value type can compile to efficient contiguous operations, while an apparently low-level pointer graph can have poor locality. Virtual dispatch, allocation, and indirection are measurable mechanisms, not inevitable properties of every class.

Keep interfaces that enforce invariants while changing internal layout. A domain-facing object can own columnar storage and expose batch operations. This

combines encapsulation with workload-oriented organization instead of forcing callers to manipulate unprotected parallel arrays.

18.2.2 Selecting Layouts Based on Workloads

Choose a layout using representative operations, data sizes, mutation rates, and deployment hardware. Evaluate correctness, implementation complexity, memory use, and performance together. A small speed gain may not justify a fragile indexing scheme or expensive migration process.

Retain a clear reference implementation and compare outputs after every representation change. A candidate that silently drops inactive-to-active transitions is not faster at the same task. Define synchronization and snapshot semantics when the data can change concurrently.

Comparing Virtual Dispatch with a Tagged Dense Representation

Purpose

Object-oriented and data-oriented layouts solve different problems. This puts dynamic polymorphism beside a dense value representation for the same square-area workload.

Code or command

```
#include <iostream>
#include <memory>
#include <vector>

struct Shape{virtual ~Shape()=default;virtual float
area()const=0;};
struct Square:Shape{float s;explicit Square(float x):s(x){}float
area()const override{return s*s;}};
struct SquareData{float side;};
int main(){
    std::vector<std::unique_ptr<Shape>>
objects;objects.push_back(std::make_unique<Square>(3));
    std::vector<SquareData> dense{{3}};
    std::cout<<objects[0]->area()<<'
'<<dense[0].side*dense[0].side<<"\n";
}
```

What the important parts do

The OO version stores a pointer to a separately allocated polymorphic object and calls through a virtual function. The dense version stores only the field required by the current transformation and performs direct arithmetic.

Expected behavior or output

Both paths print area 9.

Concrete experiment

Scale each representation to one million elements and benchmark an area sum. Then add multiple shape types and discuss when runtime polymorphism may be worth its layout and dispatch costs.

Worked example — Materialize an active processing set

Trace the implementation

ActiveSnapshot owns a materialized copy of the selected values. It does not borrow pointers into the entity vector, so later entity mutation cannot silently change the snapshot. The test makes that behavior explicit by activating a previously inactive entity and verifying that only a newly built snapshot includes it.

The direct and snapshot paths validate the same count and value domain. Repeated total calls on a snapshot can avoid repeated activity checks and validation, but that is valid only because construction established immutable accepted data. A benchmark must account for how often a new snapshot is required.

The representation intentionally omits identity because the selected operation needs only a sum. An operation that updates individual entities would need a mapping back to stable IDs and a concurrency policy. Omitting those concerns from a read-only reduction is a simplification, not a general entity-system architecture.

Complete worked source

The files below are in `source/book05/ch18/`. The source in this chapter and the companion are exported from the same authored data.

`main.cpp`

```
#include "check.hpp"
#include <cstdint>
#include <span>
#include <stdexcept>
#include <vector>
struct Entity { std::uint32_t id; bool active; int value; };
long long direct_total(std::span<const Entity> entities) {
    if (entities.size()>100000) throw std::invalid_argument("count");
    long long total=0;
    for (const auto& entity:entities) {
        if (entity.value < -1000000 || entity.value > 1000000) throw
std::invalid_argument("value");
        if (entity.active) total+=entity.value;
    }
    return total;
}
class ActiveSnapshot {
    std::vector<int> values_;
public:
    explicit ActiveSnapshot(std::span<const Entity> entities) {
        if (entities.size()>100000) throw std::invalid_argument("count");
        values_.reserve(entities.size());
        for (const auto& entity:entities) {
            if (entity.value < -1000000 || entity.value > 1000000) throw
std::invalid_argument("value");
            if (entity.active) values_.push_back(entity.value);
        }
    }
    long long total() const { long long sum=0; for (int value:values_)
sum+=value; return sum; }
    std::size_t size() const { return values_.size(); }
};
int main() {
    std::vector<Entity> entities{{1,true,8},{2,false,100},{3,true,-3}};
    const ActiveSnapshot snapshot{entities};
    CHECK(snapshot.size()==2 && snapshot.total()==5 &&
direct_total(entities)==5);
    entities[1].active=true;
    CHECK(snapshot.total()==5); // Explicit snapshot semantics.
    const ActiveSnapshot updated{entities};
    CHECK(updated.total()==105 && direct_total(entities)==105);
```

```
}    std::cout << "PASS\n";
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 18
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Benchmark one, ten, and one hundred passes per snapshot while including construction separately. Vary the active fraction and update frequency.

Extend the snapshot to retain stable IDs for reporting, then define how an update is reconciled if an entity has been removed or replaced since the snapshot was taken.

Solution and review notes

The useful crossover depends on both conversion cost and repeated-pass savings. Report the number of passes and mutation rate with any result. A snapshot that is never refreshed may be fast but semantically stale.

A stable ID lookup can detect absence, while a generation or version can detect replacement or conflicting modification. Returning a stale vector index is insufficient. Preserve these correctness checks when evaluating the performance of the extended design.

Technical references: [GUIDELINES VECTOR CPUARCH](#)

Chapter 19 — Engineering High-Performance Concurrency

Concurrency can improve throughput while also adding synchronization, queueing, and scheduling costs. A bounded task pool makes those costs and its shutdown policy explicit. Harbor’s pool returns futures for results, drains accepted work, and propagates task failures without allowing exceptions to escape worker threads.

19.1 Reducing Synchronization Costs

19.1.1 Reducing Lock Contention

Contention occurs when multiple workers need the same serialized resource. Reducing critical-section duration, batching updates, partitioning ownership, or using thread-local partial results can help. Replacing a mutex with an atomic does not automatically remove contention on the same shared cache line.

Measure the bottleneck before changing synchronization. A short uncontended mutex can be cheaper and clearer than a complicated lock-free protocol. Preserve the consistency contract: a faster approximate counter is not a drop-in replacement when callers require an exact transactional total.

19.1.2 Avoiding False Sharing

False sharing occurs when independent data used by different cores shares a coherence unit, causing traffic despite no logical data sharing. Separating frequently written per-worker state can reduce it on a measured platform. The relevant line size and topology are hardware properties, not universal constants of C++.

Padding increases memory footprint and can harm locality elsewhere. Apply it to identified hot state rather than every structure. A race-free program can still suffer false sharing, and eliminating false sharing does not prove the absence of a data race; correctness and performance are separate analyses.

Reducing Contention by Accumulating Per Thread Before One Merge

Purpose

Synchronization cost falls when workers perform independent local work and enter the shared critical section only to merge final results.

Code or command

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main(){
    std::mutex m; long long total=0; std::vector<std::jthread>
    threads;
    for(int t=0;t<4;++t) threads.emplace_back([&]{long long
    local=0;for(int i=0;i<100000;++i)++local;std::lock_guard
    lock(m);total+=local;});
    threads.clear(); std::cout<<total<<"\n";
}
```

What the important parts do

Each thread increments a private `local` counter without locking. The mutex is acquired once per thread for `total += local`, reducing lock acquisitions from 400,000 to four.

Expected behavior or output

The program prints `400000`.

Concrete experiment

Compare this design with locking around every increment. Record runtime and contention metrics while confirming both versions produce the same total.

19.2 Designing Scalable Work Distribution

19.2.1 Building Thread Pools

A pool amortizes thread creation across tasks. Its design must define queue capacity, submission behavior under load, exception propagation, shutdown, and whether workers may submit recursively. A bounded queue introduces backpressure but can deadlock a naive recursive submission pattern when all workers wait for work that cannot be admitted.

The supplied pool accepts externally submitted long-long tasks and returns futures. It is intentionally not a work-stealing runtime. Owner-thread shutdown drains accepted tasks; concurrent shutdown calls and worker-initiated shutdown are outside its contract. These boundaries are documented rather than hidden behind a generic name.

19.2.2 Applying Work Partitioning

Partition independent data so each task owns a clear range and writes disjoint output. Too-fine tasks can spend more time in queueing and synchronization than computation; too-coarse tasks can leave workers idle when work is uneven. Choose granularity using measured workload cost and variability.

Keep deterministic correctness tests separate from scheduling assumptions. Completion order may differ from submission order, but associating each future with its task preserves result identity. A reduction can combine results after completion without requiring all tasks to finish in a particular order.

Static Work Partitioning Across a Fixed Thread Team

Purpose

Work distribution should give each worker enough independent work to amortize scheduling overhead while avoiding shared writes in the hot path.

Code or command

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <thread>
#include <vector>

int main(){
    std::vector<int> data(1'000'000,1); constexpr int workers=4; long
    long partial[workers]{}; std::vector<std::jthread> ts;
    for(int w=0;w<workers;++w) ts.emplace_back([&,w]{auto
    first=data.begin()+w*(data.size()/workers);auto
    last=(w==workers-1)?data.end():first+(data.size()/
    workers);partial[w]=std::accumulate(first,last,0LL);});
    ts.clear();
    std::cout<<std::accumulate(std::begin(partial),std::end(partial),0LL)
    <<"\n";
}
```

What the important parts do

Each worker receives a deterministic contiguous slice and stores its result in a separate array slot. Joining occurs before the final reduction over four partial sums.

Expected behavior or output

The program prints `1000000`.

Concrete experiment

Change the input so the cost per element is highly uneven. Observe load imbalance under static partitioning, then design a chunk queue or work-stealing approach and compare.

Worked example — Build a bounded future-returning task pool

Trace the implementation

The shared `bounded_queue.hpp` is the fully supplied queue from Book IV. Each submitted operation is wrapped in `packaged_task`, which stores either its result or its exception in the future's shared state. The worker invokes a valid `packaged`

task, so a user-operation exception is retrieved by the consumer rather than escaping the thread entry point.

A `shared_ptr` makes the wrapper copyable for `std::function`, even though `packaged_task` itself is move-only. That convenience has allocation and reference-counting costs. The example exposes those costs rather than calling the pool zero-overhead. A production executor may use a move-only task wrapper or preallocated storage after measurement justifies it.

Constructor failure closes the queue before joining already-created workers. `shutdown` closes admission and drains queued tasks before clearing the `jthread` vector. Calling `shutdown` twice from the owner thread is harmless. Calling it from a worker would attempt to join the current thread and is explicitly prohibited by the interface contract.

Complete worked source

The files below are in `source/book05/ch19/`. The source in this chapter and the companion are exported from the same authored data.

`pool.hpp`

```
#ifndef HARBOR_TASK_POOL_HPP
#define HARBOR_TASK_POOL_HPP
#include "bounded_queue.hpp"
#include <functional>
#include <future>
#include <memory>
#include <stdexcept>
#include <thread>
#include <vector>
class TaskPool {
    BoundedQueue<std::function<void()>> work_;
    std::vector<std::jthread> workers_;
public:
    explicit TaskPool(std::size_t workers, std::size_t
capacity=8):work_(capacity) {
        if (workers==0 || workers>64) throw std::invalid_argument("worker
count");
        try {
            for (std::size_t i=0; i<workers; ++i)
                workers_.emplace_back([this] { while (auto task=work_.pop())
(*task)(); });
        } catch (...) { work_.close(); workers_.clear(); throw; }
        TaskPool(const TaskPool&)=delete;
        TaskPool& operator=(const TaskPool&)=delete;
        ~TaskPool() { shutdown(); }
        std::future<long long> submit(std::function<long long()> operation) {
            if (!operation) throw std::invalid_argument("empty task");
            auto task=std::make_shared<std::packaged_task<long
long()>>(std::move(operation));
            auto result=task->get_future();
            if (!work_.push([task] { (*task)(); })) throw std::runtime_error("pool
closed");
            return result;
        }
        // Owner-thread only; never call from a worker or concurrently with
shutdown.
        void shutdown() { work_.close(); workers_.clear(); }
```



```
};  
#endif
```

main.cpp

```
#include "check.hpp"  
#include "pool.hpp"  
#include <future>  
#include <stdexcept>  
#include <vector>  
int main() {  
    TaskPool pool{3,4};  
    std::vector<std::future<long long>> results;  
    for (int value=0;value<20;++value)  
        results.push_back(pool.submit([value] { return static_cast<long  
long>(value)*value; }));  
    for (int value=0;value<20;++value)  
        CHECK(results[static_cast<std::size_t>(value)].get()==value*value);  
    auto failed=pool.submit([]()->long long { throw std::runtime_error("planned  
task failure"); });  
    bool propagated=false;  
    try { (void)failed.get(); } catch (const std::runtime_error&)  
{ propagated=true; }  
    CHECK(propagated);  
    pool.shutdown();  
    bool closed=false;  
    try { (void)pool.submit([] { return 1LL; }); } catch (const  
std::runtime_error&) { closed=true; }  
    CHECK(closed);  
    std::cout << "PASS\n";  
}
```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 19
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Partition a bounded input vector into tasks that return partial sums, then compare the combined result with a serial reference. Vary task size while recording queueing and end-to-end time.

Add a nonblocking submission API with distinct full and closed outcomes. Explain how recursive worker submission could deadlock the current blocking design.

Solution and review notes

A worker that blocks trying to submit into a full queue can occupy a thread needed to drain that queue. If every worker does so, no progress is possible. Work stealing,

inline execution, reserved capacity, or a prohibition on recursive submission are different policies with different guarantees.

The nonblocking result should preserve ownership when a task is not accepted and should not confuse temporary capacity pressure with permanent closure. Test the state transition and result propagation before benchmarking it.

Technical references: [THREADS](#) [CONDVAR](#) [FUTURE](#) [MEMORYMODEL](#)

Chapter 20 — Designing Stable Libraries and APIs

An API communicates what callers may depend on; an ABI describes how compiled components interact. A source-compatible change can still break an old binary, and a stable-looking class name does not guarantee a stable layout. Harbor’s accumulator uses Pimpl to hide implementation storage while keeping its move and failure behavior explicit.

20.1 Designing Clear Public Interfaces

20.1.1 Minimizing Coupling

Expose the operations and domain types consumers need, not every implementation detail available. A smaller public surface reduces the number of assumptions that later changes can violate. Avoid leaking third-party containers or platform handles unless they are intentionally part of the contract.

Clarity includes ownership, error behavior, thread safety, and invalid-state policy. A short signature with undocumented lifetime requirements is not a simple API. State whether returned data is owned or borrowed and whether concurrent calls on one instance are supported.

20.1.2 Preserving Encapsulation

Encapsulation protects invariants by controlling how state changes. It does not mean hiding all information from tests; expose meaningful observations that let callers verify behavior without depending on internal representation. A count and total can be stable observations even when the storage strategy changes.

The Pimpl idiom stores a pointer to an incomplete implementation type in the public class and defines operations where that type is complete. This can reduce header dependencies and hide layout details of the implementation. The public object’s own representation and special-member behavior still require a compatibility policy.

Exposing a Minimal Public API While Hiding Representation

Purpose

A clear public interface states supported operations without exposing containers, mutexes, logging systems, or other implementation choices that would couple every consumer to internal design.

Code or command

```
// counter.hpp
#pragma once
#include <memory>
class Counter{
public:
    Counter(); ~Counter();
    Counter(Counter&&) noexcept; Counter& operator=(Counter&&)
    noexcept;
    void increment(); int value() const;
private:
    struct Impl; std::unique_ptr<Impl> impl_;
};
```

What the important parts do

The header declares behavior and ownership semantics while an incomplete `Impl` type hides representation. Consumers can compile against this header without seeing the private fields that implement the counter.

Expected behavior or output

Code using `increment` and `value` remains source-compatible when the hidden implementation changes but the public contract does not.

Concrete experiment

Implement `Impl` first with an `int`, then with `std::atomic<int>`. Confirm consumer source files need no edits and analyze which binary-compatibility conditions still depend on the library build.

20.2 Managing Binary Compatibility

20.2.1 Understanding ABI Concerns

ABI compatibility can depend on compiler family, standard-library implementation, runtime settings, calling convention, exception model, symbol names, and public type layout. Rebuilding every component together tests source integration, not compatibility with already-shipped consumers.

Define the supported binary boundary explicitly. Some products rebuild consumers for every release; others maintain a stable C interface or a versioned shared-library ABI. Do not promise universal binary compatibility merely because a function uses familiar standard-library types.

20.2.2 Applying the Pimpl Idiom

A `unique_ptr` to an incomplete `Impl` can appear in a public header, but destruction and relevant special members should be defined where `Impl` is complete. This avoids requiring clients to instantiate deletion against an incomplete type. Move operations also need a stated moved-from policy.

Pimpl adds indirection and usually a separate allocation. It can improve build isolation and permit internal changes within a controlled ABI environment, but it does not automatically stabilize the C++ runtime or compiler ABI. Evaluate the tradeoff against header-only optimization and deployment requirements.

Using Pimpl to Keep Private Layout Out of the Public Class

Purpose

ABI stability is affected by public object size and layout. Pimpl fixes the public object largely to the size/layout of an owning pointer while moving volatile private members into separately allocated storage.

Code or command

```
// widget.hpp
#pragma once
#include <memory>
class Widget{
public:
    Widget(); ~Widget();
    Widget(Widget&&) noexcept; Widget& operator=(Widget&&) noexcept;
    int compute() const;
private:
    class Impl;
    std::unique_ptr<Impl> p_;
};
```

What the important parts do

Only the pointer member appears in the class definition. Adding fields to `Impl` in the `.cpp` file does not directly change `sizeof(Widget)`. The out-of-line destructor is needed because `Impl` is incomplete in the header.

Expected behavior or output

A consumer can observe a stable `sizeof(Widget)` across private implementation changes, subject to compiler/standard-library ABI compatibility.

Concrete experiment

Record `sizeof(Widget)` before and after adding several fields to `Impl`. Then add a public data member to `Widget` itself and observe the ABI-visible size change.

Worked example — Hide accumulator storage behind Pimpl

Trace the implementation

The public header contains only the incomplete `Impl` declaration and an owning pointer. The destructor and move operations are defined in `accumulator.cpp` after `Impl` is complete. Consumers compile against a stable source-level declaration without seeing the current count/total representation.

After a move, the source may be destroyed or assigned a fresh accumulator, but calling its observation or mutation functions throws a documented logic error. The test checks both rejection and reinitialization. This is more precise than vaguely declaring that a moved-from object remains usable without saying which operations are supported.

The count and value bounds prove the total fits in long long. Internal storage could later change to retain samples or use a different aggregation strategy while preserving these public observations. That source-level flexibility is not, by itself, a cross-compiler ABI guarantee. A release policy must still specify compatible toolchains and binary expectations.

Complete worked source

The files below are in `source/book05/ch20/`. The source in this chapter and the companion are exported from the same authored data.

`accumulator.hpp`

```
#ifndef HARBOR_ACCUMULATOR_HPP
#define HARBOR_ACCUMULATOR_HPP
#include <stddef>
#include <memory>
namespace harbor {
class Accumulator {
    struct Impl;
    std::unique_ptr<Impl> impl_;
public:
    Accumulator();
    ~Accumulator();
    Accumulator(Accumulator&&) noexcept;
    Accumulator& operator=(Accumulator&&) noexcept;
    Accumulator(const Accumulator&)=delete;
    Accumulator& operator=(const Accumulator&)=delete;
    void add(int value);
    long long total() const;
    std::size_t count() const;
};
}
#endif
```

`accumulator.cpp`

```
#include "accumulator.hpp"
#include <stdexcept>
namespace harbor {
struct Accumulator::Impl { long long total{}; std::size_t count{}; };
Accumulator::Accumulator():impl_(std::make_unique<Impl>()) {}
Accumulator::~Accumulator()=default;
Accumulator::Accumulator(Accumulator&&) noexcept=default;
Accumulator& Accumulator::operator=(Accumulator&&) noexcept=default;
void Accumulator::add(int value) {
    if (!impl_) throw std::logic_error("moved-from accumulator");
    if (value < -1000000 || value > 1000000 || impl_->count==1000000)
        throw std::invalid_argument("accumulator bounds");
    impl_->total+=value; ++impl_->count;
}
long long Accumulator::total() const {
    if (!impl_) throw std::logic_error("moved-from accumulator");
```

```

        return impl_>total;
    }
    std::size_t Accumulator::count() const {
        if (!impl_) throw std::logic_error("moved-from accumulator");
        return impl_>count;
    }
}

```

main.cpp

```

#include "check.hpp"
#include "accumulator.hpp"
#include <stdexcept>
#include <utility>
int main() {
    harbor::Accumulator first; first.add(7); first.add(-2);
    harbor::Accumulator second=std::move(first);
    CHECK(second.total()==5 && second.count()==2);
    bool rejected=false;
    try { (void)first.total(); } catch (const std::logic_error&)
    { rejected=true; }
    CHECK(rejected);
    first=harbor::Accumulator{}; first.add(3);
    CHECK(first.total()==3);
    std::cout << "PASS\n";
}

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 20
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Change Impl to retain a vector of accepted values while preserving count and total behavior. Rebuild the library and consumer separately in a controlled compatible environment, then distinguish source compatibility from an actual old-consumer ABI test.

Add a copy operation with an explicit deep-copy contract, defining it out of line where Impl is complete.

Solution and review notes

A deep copy allocates an independent Impl and copies its state; copying the raw implementation pointer would create shared ownership or double deletion. Copy assignment should construct a candidate before committing when a strong exception guarantee is required.

To test binary compatibility, retain an old compiled consumer and replace only the library within the declared supported ABI environment. Recompiling the consumer against the new header answers a different question. Preserve both artifact identities in the result.

Technical references: [MEMORY GUIDELINES](#) [ABI ODR](#)

Chapter 21 — Building, Testing, Profiling, and Shipping a Production System

HarborMetrics turns a bounded measurement file into a deterministic summary. Its release workflow builds the application, runs contract tests, stages an installation, records artifact hashes, and creates a distributable archive. This is a complete small application and a reproducible packaging exercise, not a claim that every operational or regulatory requirement of a real organization has been satisfied.

The final engineering decision is evidence-based: a candidate implementation must preserve the result and error contract before its timing is considered. A release is then identified by the source, toolchain, configuration, tests, and packaged bytes that actually produced it.

21.1 Establishing the Production Build Pipeline

21.1.1 Automating Compilation and Tests

A pipeline should fail at the stage where evidence becomes invalid. Configuration failure must stop compilation; compilation failure must stop testing and packaging; failed tests must prevent a success-labeled release artifact. Preserve logs and exit status rather than printing a reassuring message after an ignored subprocess failure.

Use clean build directories for distinct configurations and keep dependencies explicit. The supplied release script invokes CMake and CTest with checked subprocess calls. Its packaging path is separate from the source tree, and it refuses to reuse an existing destination that might contain stale artifacts.

21.1.2 Producing Release Artifacts

A release artifact contains the files needed by its intended consumer, plus enough identity information to diagnose it. For a native executable, platform and runtime compatibility still matter even when the archive itself is portable as a ZIP file. Document how the binary was built and which external runtime dependencies remain.

Hashes identify bytes but are not signatures or proof of trust. A production distribution process may additionally sign artifacts, generate a software bill of materials, retain symbols, scan dependencies, and enforce approval. The teaching script records actual staged hashes without claiming those additional controls are present.

Production Pipeline that Builds, Tests, and Packages One Immutable Revision

Purpose

A production pipeline should establish one chain from revision to tested artifact, with packaging performed only after the same build passes required tests.

Code or command

```
steps:
- run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
- run: cmake --build build --parallel
- run: ctest --test-dir build --output-on-failure
- run: cmake --install build --prefix stage
- run: tar -C stage -czf app-${GIT_SHA}.tar.gz .
- run: sha256sum app-${GIT_SHA}.tar.gz > app-${GIT_SHA}.sha256
```

What the important parts do

The steps configure and compile Release output, execute CTest, stage install rules, package the staged tree with the revision in its name, and produce a checksum for artifact identity.

Expected behavior or output

Success produces a versioned archive and matching checksum only after tests complete successfully.

Concrete experiment

Force one test to fail and verify packaging does not run in your CI system. Then publish the checksum and source revision as artifact metadata.

21.2 Optimizing Based on Evidence

21.2.1 Establishing Performance Baselines

Define the workload, accepted input domain, numerical contract, and measurement boundary before changing code. A baseline includes errors and correctness checks, not only a throughput number. Keep the original implementation available as an independent comparison while evaluating a candidate.

HarborMetrics compares a straightforward total with a four-lane reduction after the same validation pass. Both produce the same bounded integer total, extrema, count, and derived mean. The benchmark reports actual elapsed durations and

checks equality; it never assumes that the more elaborate implementation must be faster.

21.2.2 Verifying Measured Improvements

A candidate is acceptable only if its improvement survives representative inputs and does not violate resource, accuracy, compatibility, or maintainability requirements. Repeat measurements, inspect variance, and test the release configuration on the intended hardware. A local microbenchmark is not a service-level performance guarantee.

Retain the evidence with the artifact. If an optimization helps only one target, a supported dispatch or configuration policy may be better than replacing every implementation. Reject unsubstantiated complexity when the measured benefit is absent or too small for the maintenance cost.

Accepting an Optimization Only After Baseline and Correctness Comparison

Purpose

Evidence-based optimization keeps a known-correct reference and treats performance as a separate question after functional equivalence is established.

Code or command

```
#include <algorithm>
#include <cassert>
#include <numeric>
#include <vector>

long long baseline(const std::vector<int>& v){return
std::accumulate(v.begin(),v.end(),0LL);}
long long candidate(const std::vector<int>& v){long long
a=0,b=0;for(std::size_t i=0;i<v.size();++i)(i&1?b:a)+=v[i];return
a+b;}
int main(){std::vector<int>
v(100000,3);assert(candidate(v)==baseline(v));}
```

What the important parts do

`baseline` uses a simple standard algorithm. `candidate` changes the dependency structure with two accumulators. The assertion prevents a faster but wrong implementation from being accepted as an optimization.

Expected behavior or output

The program exits successfully because both implementations return the same sum for the fixture.

Concrete experiment

Benchmark both functions over multiple data sizes after the equality test. Report distributions and generated assembly, then keep the candidate only if measured improvement justifies added complexity.

Worked example — Build and package HarborMetrics

Trace the implementation

The application accepts files without a header, one tick,value record per line. Ticks must be nondecreasing and at most one trillion; values must lie between minus one million and one million; the file contains one through 100,000 records. Lines are bounded to sixty-four bytes during reading. Blank records, malformed fields,

excessive values, and out-of-order timestamps are rejected before a report is published.

Run the chapter executable with `-input` and the supplied `example.csv` to obtain the exact report checked by the self-test. `-version` prints the application version, while `-benchmark` compares the two reduction paths. The summary's bounded integer total is exact; its printed mean is rounded to three decimal places by the reporting layer.

The optional release script is the actual build-and-package implementation. From the package root, run `python3 source/book05/ch21/optional/release.py /path/to/new/release-directory`, choosing a new writable directory. It refuses an existing destination, retains a `build/test/install` log, hashes the staged executable, and creates a ZIP. It does not claim to produce one binary that works on every operating system or CPU.

Before real deployment, add the environment-specific requirements: dependency and license inventory, supported runtime baseline, code signing or trusted distribution, vulnerability review, retained symbols, operational monitoring, and rollback. These are explicit readiness items, not properties implied by a successful CMake install.

Complete worked source

The files below are in `source/book05/ch21/`. The source in this chapter and the companion are exported from the same authored data.

`metrics.hpp`

```
#ifndef HARBOR_METRICS_HPP
#define HARBOR_METRICS_HPP
#include <cstdint>
#include <cstdint>
#include <iosfwd>
#include <span>
#include <vector>
namespace harbor {
    inline constexpr const char* version="0.1.0";
    struct Record { std::uint64_t tick; int value; };
    struct Summary { std::size_t count; long long total; int minimum,maximum; double mean; };
    std::vector<Record> read_records(std::istream& input);
    Summary analyze(std::span<const Record> records,bool four_lanes=false);
    void write_report(std::ostream& output,const Summary& summary);
}
#endif
```

`metrics.cpp`

```
#include "metrics.hpp"
#include <algorithm>
#include <array>
#include <charconv>
#include <iomanip>
#include <istream>
#include <ostream>
```

```

#include <stdexcept>
#include <string>
#include <string_view>
namespace harbor {
namespace {
constexpr std::size_t maximum_records=100000;
Record parse_line(std::string_view line) {
    if (!line.empty() && line.back()=='\r') line.remove_suffix(1);
    const auto comma=line.find(',');
    if (comma==std::string_view::npos || line.find(',',comma+1)!
=std::string_view::npos)
        throw std::runtime_error("expected tick,value");
    Record result{};
    auto tick=line.substr(0,comma), value=line.substr(comma+1);
    const auto [a,ae]=std::from_chars(tick.data(),tick.data()
+tick.size(),result.tick);
    const auto [b,be]=std::from_chars(value.data(),value.data()
+value.size(),result.value);
    if (ae!=std::errc{} || a!=tick.data()+tick.size() || be!=std::errc{} || b!
=value.data()+value.size())
        throw std::runtime_error("invalid numeric field");
    return result;
}
void validate(std::span<const Record> records) {
    if (records.empty() || records.size()>maximum_records) throw
std::runtime_error("record count");
    std::uint64_t previous=0;
    for (const auto& record:records) {
        if (record.tick>1000000000000ULL || record.tick<previous)
            throw std::runtime_error("timestamp order or range");
        if (record.value < -1000000 || record.value > 1000000)
            throw std::runtime_error("value range");
        previous=record.tick;
    }
}
}
std::vector<Record> read_records(std::istream& input) {
    std::vector<Record> records;
    std::string line;
    auto append=[&] {
        if (records.size()==maximum_records) throw std::runtime_error("record
count");
        records.push_back(parse_line(line)); line.clear();
    };
    char character{};
    while (input.get(character)) {
        if (character=='\n') append();
        else {
            if (line.size()==64) throw std::runtime_error("line length");
            line.push_back(character);
        }
    }
    if (!input.eof()) throw std::runtime_error("input read failure");
    if (!line.empty()) append();
    validate(records);
    return records;
}
Summary analyze(std::span<const Record> records,bool four_lanes) {
    validate(records);
    long long total=0;
    if (four_lanes) {
        std::array<long long,4> partial{};
        std::size_t i=0;
        for (;i+4<=records.size();i+=4)
            for (std::size_t lane=0;lane<4;++lane) partial[lane]
+=records[i+lane].value;
    }
}

```

```

        for (auto value:partial) total+=value;
        for (;i<records.size();++i) total+=records[i].value;
    } else {
        for (const auto& record:records) total+=record.value;
    }
    int minimum=records.front().value, maximum=minimum;
    for (const auto& record:records) {
        minimum=std::min(minimum,record.value);
    }
    maximum=std::max(maximum,record.value);
    return {records.size(),total,minimum,maximum,static_cast<double>(total)/
    static_cast<double>(records.size())};
}
void write_report(std::ostream& output,const Summary& summary) {
    output << "count=" << summary.count << " total=" << summary.total
        << " min=" << summary.minimum << " max=" << summary.maximum
        << " mean=" << std::fixed << std::setprecision(3) << summary.mean <<
    '\n';
    if (!output) throw std::runtime_error("report write failure");
}
}

```

main.cpp

```

#include "check.hpp"
#include "metrics.hpp"
#include <chrono>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <string_view>
#include <vector>
void self_test() {
    std::istringstream input{"1,5\n2,-2\n3,9\n"};
    const auto records=harbor::read_records(input);
    const auto baseline=harbor::analyze(records),
    candidate=harbor::analyze(records,true);
    CHECK(baseline.total==12 && baseline.count==3 && baseline.mean==4.0);
    CHECK(candidate.total==baseline.total && candidate.minimum==-2 &&
    candidate.maximum==9);
    std::ostringstream output; harbor::write_report(output,baseline);
    CHECK(output.str()=="count=3 total=12 min=-2 max=9 mean=4.000\n");
    for (const char* text:{"", "2,4\n1,5\n", "1,2x\n", "1,1000001\n",
    "1,2,3\n"}) {
        std::istringstream bad{text}; bool rejected=false;
        try { (void)harbor::read_records(bad); } catch (const
    std::runtime_error&) { rejected=true; }
        CHECK(rejected);
    }
    std::cout << "PASS\n";
}
void benchmark() {
    std::vector<harbor::Record> records;
    for (std::uint64_t i=0;i<10003;++i)
    records.push_back({i,static_cast<int>(i%101)-50});
    for (bool lanes:{false,true}) {
        long long checksum=0;
        const auto start=std::chrono::steady_clock::now();
        for (int round=0;round<20;++round) {
            records[0].value=round;
            checksum+=harbor::analyze(records,lanes).total;
        }
        const auto
    elapsed=std::chrono::duration<double,std::milli>(std::chrono::steady_clock::now(
    )-start).count();
}

```



```

        std::cout << "four_lanes=" << lanes << " elapsed_ms=" << elapsed << "
checksum=" << checksum << '\n';
    }
    CHECK(harbor::analyze(records).total==harbor::analyze(records,true).total);
}
int main(int argc,char** argv) {
    try {
        if (argc==1 || (argc==2 && std::string_view(argv[1])=="--self-test"))
        { self_test(); return 0; }
        if (argc==2 && std::string_view(argv[1])=="--version") { std::cout <<
harbor::version << '\n'; return 0; }
        if (argc==2 && std::string_view(argv[1])=="--benchmark") { benchmark();
return 0; }
        if (argc==3 && std::string_view(argv[1])=="--input") {
            std::ifstream input(argv[2],std::ios::binary);
            if (!input) throw std::runtime_error("cannot open input");
            const auto records=harbor::read_records(input);
            harbor::write_report(std::cout,harbor::analyze(records));
            return 0;
        }
        std::cerr << "usage: harbor-metrics [--self-test|--version|--
benchmark|--input FILE]\n";
        return 2;
    } catch (const std::exception& error) { std::cerr << error.what() << '\n';
return 1; }
}

```

example.csv

```

1,5
2,-2
3,9

```

Optional source: optional/CMakeLists.txt. This file has a separately stated build or execution procedure and is not part of the default build.

```

cmake_minimum_required(VERSION 3.24)
project(HarborMetrics VERSION 0.1.0 LANGUAGES CXX)
include(CTest)
add_library(metrics STATIC ../metrics.cpp)
target_compile_features(metrics PUBLIC cxx_std_20)
target_include_directories(metrics PUBLIC "${CMAKE_CURRENT_LIST_DIR}/..")
add_executable(harbor-metrics ../main.cpp)
target_include_directories(harbor-metrics PRIVATE "$
{CMAKE_CURRENT_LIST_DIR}/../../support")
target_link_libraries(harbor-metrics PRIVATE metrics)
set_target_properties(metrics harbor-metrics PROPERTIES CXX_EXTENSIONS OFF)
add_test(NAME self_test COMMAND harbor-metrics --self-test)
add_test(NAME file_contract COMMAND harbor-metrics --input "$
{CMAKE_CURRENT_LIST_DIR}/../example.csv")
set_tests_properties(file_contract PROPERTIES PASS_REGULAR_EXPRESSION "count=3
total=12 min=-2 max=9 mean=4.000")
install(TARGETS harbor-metrics RUNTIME DESTINATION bin)

```

Optional source: optional/release.py. This file has a separately stated build or execution procedure and is not part of the default build.

```

#!/usr/bin/env python3
"""Build, test, stage, hash, and archive the real application. No network
access."""
from pathlib import Path

```

```

import argparse
import hashlib
import json
import platform
import shutil
import subprocess
parser=argparse.ArgumentParser()
parser.add_argument("destination",type=Path,help="new directory; existing paths
are refused")
args=parser.parse_args()
out=args.destination.resolve()
out.mkdir(parents=True,exist_ok=False)
source=Path(__file__).resolve().parent
build=out/"build"
stage=out/"stage"
commands=[
    ["cmake","-S",str(source),"-B",str(build),"-DCMAKE_BUILD_TYPE=Release"],
    ["cmake","--build",str(build),"--config","Release","--parallel","2"],
    ["ctest","--test-dir",str(build),"-C","Release","--output-on-failure"],
    ["cmake","--install",str(build),"--config","Release","--prefix",str(stage)],
]
with (out/"release.log").open("w",encoding="utf-8") as log:
    for command in commands:
        log.write("COMMAND: "+repr(command)+"\n"); log.flush()

subprocess.run(command,check=True,stdout=log,stderr=subprocess.STDOUT,text=True)
manifest={"application":"HarborMetrics","version":"0.1.0","platform":platform.pl
atform(),"files":[]}
for path in sorted(stage.rglob("*")):
    if path.is_file():
        manifest["files"].append({"path":str(path.relative_to(stage)),"sha256":hashlib.s
ha256(path.read_bytes()).hexdigest()})
(stage/"manifest.json").write_text(json.dumps(manifest,indent=2)+"\n",encoding="
utf-8")
archive=shutil.make_archive(str(out/"harbor-metrics-0.1.0"),"zip",stage)
print(archive)

```

Run and verify

From the extracted package root:

```
python3 tools/run_chapter.py 5 21
```

The runner configures the companion, builds this chapter, and executes its fixture tests. A failed check returns a nonzero status; release builds do not disable the checks.

Practice and implementation lab

Add full end-to-end cases for CRLF input, a final record without a newline, maximum accepted values, and an oversized line. Verify exact stdout, stderr, and exit status at the CLI boundary.

Run the release script in a clean directory, inspect the staged manifest, and test the installed executable rather than only the build-tree executable. Compare the two analysis implementations across several record counts and retain raw timing samples.

Solution and review notes

CRLF is handled by removing one trailing carriage return from a line; the final nonempty line is parsed at EOF. A blank line is still an invalid record. An oversized line fails during bounded accumulation, preventing a huge token from consuming arbitrary memory before validation.

A release check should hash and execute the actual staged artifact. If it needs external shared libraries, inspect and document them on the target platform. A measured candidate that does not consistently improve the representative workload should not replace the clearer baseline merely because its loop appears more sophisticated.

Technical references: [CMAKE](#) [CMAKETARGET](#) [CHARCONV](#) [CHRONO](#) [BENCHMARK](#)

Glossary

ABI. The binary-level contract between compiled components, including calling and representation conventions.

Benchmark boundary. The exact operations included in a measured interval.

Compilation database. A record of per-translation-unit compiler commands used by development tools.

False sharing. Coherence traffic caused by independent data sharing a hardware coherence unit.

Golden fixture. An independently specified expected representation used to check a boundary.

Pimpl. A technique that hides implementation state behind a pointer to an incomplete private type.

Reproducibility. The ability to reconstruct a stated result from identified inputs and procedures.

SIMD. Single-instruction, multiple-data execution over several lanes.

Usage requirement. Build information a target publishes to consumers, such as language features or include paths.

Shared source reference

These headers are included by the chapter programs. Their complete text is provided here so the book does not hide essential ownership, checking, or platform logic. The companion stores them under `source/support/`. Include that directory when compiling manually; the supplied CMake targets do so automatically.

check.hpp

```
#ifndef HARBOR_SERIES_CHECK_HPP
#define HARBOR_SERIES_CHECK_HPP
#include <iostream>
#include <source_location>
#include <stdexcept>
#include <string>

namespace harbor_test {
inline void check(bool passed, const char* expression,
                  std::source_location location =
std::source_location::current()) {
    if (!passed) {
        throw std::runtime_error(std::string(location.file_name()) + ":" +
                                std::to_string(location.line()) + ": check failed: " + expression);
    }
}
}
#define CHECK(...) ::harbor_test::check(static_cast<bool>((__VA_ARGS__)),
__VA_ARGS__)
#endif
```

bounded_queue.hpp

```
#ifndef HARBOR_SERIES_BOUNDED_QUEUE_HPP
#define HARBOR_SERIES_BOUNDED_QUEUE_HPP
#include <condition_variable>
#include <cstdint>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <utility>

template<class T>
class BoundedQueue {
    static_assert(std::is_nothrow_move_constructible_v<T>);
    std::mutex mutex_;
    std::condition_variable not_empty_, not_full_;
    std::deque<T> data_;
    std::size_t capacity_;
    bool closed_{};
};
```

```

public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity) {
        if (capacity == 0) throw std::invalid_argument("queue capacity");
    }
    BoundedQueue(const BoundedQueue&) = delete;
    BoundedQueue& operator=(const BoundedQueue&) = delete;
    bool push(T value) {
        std::unique_lock lock{mutex_};
        not_full_.wait(lock, [&] { return closed_ || data_.size() <
capacity_; });
        if (closed_) return false;
        data_.push_back(std::move(value));
        lock.unlock();
        not_empty_.notify_one();
        return true;
    }
    std::optional<T> pop() {
        std::unique_lock lock{mutex_};
        not_empty_.wait(lock, [&] { return closed_ || !data_.empty(); });
        if (data_.empty()) return std::nullopt;
        T value = std::move(data_.front());
        data_.pop_front();
        lock.unlock();
        not_full_.notify_one();
        return value;
    }
    void close() {
        { std::lock_guard lock{mutex_}; closed_ = true; }
        not_empty_.notify_all();
        not_full_.notify_all();
    }
};
#endif

```

Technical reference directory

The numbered curriculum and its learning goals come from the supplied series outline. The explanations, fictional Harbor applications, experiments, and exercises are original teaching material. Primary project documentation below supports the language and platform mechanisms. The Linux book supplied the editorial pattern, not C++ subject matter. Documentation was consulted for this 2026 edition; rolling pages can change after publication.

The ISO working-draft mirror is a living draft, not a claim that every feature it documents exists in C++20. Match language rules, compiler support, library support, and toolkit documentation to the actual versions used for a build. No proprietary standard text or vendor manual is reproduced.

ABI — Itanium C++ ABI

[Itanium C++ ABI](#)

An implementation ABI specification, not a universal language or Windows ABI guarantee.

ALGORITHMS — Algorithms library

[Algorithms library](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

ASAN — Clang AddressSanitizer

[Clang AddressSanitizer](#)

Instrumented memory-error detection, configuration, and limitations.

BENCHMARK — Google Benchmark user guide

[Google Benchmark user guide](#)

Benchmark methodology and optimization barriers; the worked programs also show dependency-free timing loops.

CHARCONV — Primitive numeric conversions

[Primitive numeric conversions](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CHRONO — Time library

[Time library](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CLANG — Clang C++ language support

[Clang C++ language support](#)

Compiler feature status; check the standard library separately.

CLANGTIDY — clang-tidy documentation

[clang-tidy documentation](#)

Static-analysis checks and compilation-database integration.

CMAKE — CMake tutorial

[CMake tutorial](#)

Target-oriented configure, build, test, and install concepts. Latest documentation may describe newer CMake features than the 3.24 project minimum.

CMAKETARGET — CMake target_link_libraries

[CMake target link libraries](#)

Target dependencies and PUBLIC, PRIVATE, and INTERFACE usage requirements.

CONDVAR — Condition variables

[Condition variables](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

CPUARCH — Intel optimization manuals

[Intel optimization manuals](#)

Processor-specific architecture and optimization references; no universal cache size or cycle latency is assumed.

DEPENDENCIES — CMake FetchContent

[CMake FetchContent](#)

Dependency acquisition and project integration; the default companion build does not fetch third-party code.

FILESYSTEM — Filesystem library

[Filesystem library](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

FUTURE — Futures and promises

[Futures and promises](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

GCC — GCC C++ language support

[GCC C++ language support](#)

Compiler support varies by version, language mode, and library.

GDB — Debugging with GDB

[Debugging with GDB](#)

Breakpoints, watchpoints, stack frames, and core-file analysis.

GIT — Git documentation

[Git documentation](#)

Version control commands and repository behavior.

GUIDELINES — C++ Core Guidelines

[C++ Core Guidelines](#)

Design guidance on ownership, interfaces, safety, and concurrency; recommendations are not language rules.

INTRINSICS — Intel Intrinsics Guide

[Intel Intrinsics Guide](#)

Instruction-set requirements and intrinsic operations; choose a compatible target or runtime dispatch.

IO — Input/output library

[Input/output library](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

LIFETIME — Object lifetime

[Object lifetime](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

LLDB — LLDB tutorial

[LLDB tutorial](#)

Debugger command concepts and native-program inspection.

MEMORY — Memory management utilities

[Memory management utilities](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

MEMORYMODEL — Data races and happens-before

[Data races and happens-before](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

NUMERIC — Fundamental types and numeric limits

[Fundamental types and numeric limits](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

ODR — One-definition rule

[One-definition rule](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

OPTIMIZE — GCC optimization options

[GCC optimization options](#)

Optimization levels and flags; exact behavior depends on compiler and target.

PERF — perf stat manual

[perf stat manual](#)

Linux hardware-counter collection; availability and permissions vary.

SPAN — Non-owning span

[Non-owning span](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

THREADS — Threads and concurrency

[Threads and concurrency](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

UBSAN — Clang UndefinedBehaviorSanitizer

[Clang UndefinedBehaviorSanitizer](#)

Selected runtime undefined-behavior checks; absence of a diagnostic is not a correctness proof.

VECTOR — Vector container

[Vector container](#)

Living working-draft reference. Check revision-specific wording against the chosen C++ language mode; later additions are not automatically C++20 features.

VECTORIZER — LLVM auto-vectorization

[LLVM auto-vectorization](#)

Loop/SLP vectorization and diagnostics; inspect the actual compiler result.